Virtio ENV 源码导读

从每个对象的创建、队列分配到 TX/RX 业务闭环

版本：1.0 · 代码基线 e2a0a46

更新日期：2026-10-10

阅读主线：对象创建 → 资源建立 → 业务启动 → 结果检查

目录

1. 阅读范围与方法
 1.1 本文从哪里开始，到哪里结束
 1.2 先区分六种动作
 1.3 两条路径不能混讲
 1.4 默认主线参数与地址记法
 1.5 目录和层次分工
2 从正式测试入口认识对象树
 2.1 阅读基准与第一条调用链
 2.2 fixture 先准备什么，为什么必须在 env 之前
 2.3 execution profile 与 MODEL 附件
3 virtio_net_env 的每个成员从哪里来
 3.1 配置与权威快照：借用外部，创建本地只读上下文
 3.2 Host memory、IOMMU 与共享策略
 3.3 PF、Function、VF 数组并不是三套设备
 3.4 验证、性能、并发与动态重配置成员
4 UVM phase 如何把对象变成可执行流程
 4.1 build：先有拓扑身份，再有 Function 内部对象
 4.2 connect：先装行为和 analysis，再由 fixture 接 PCIe
 4.3 PCIe 绑定的预检、提交与失败边界
 4.4 run 与 report：env 没有自动初始化线程
5 Function 内部：创建、身份应用、协议选择与延迟建队列
 5.1 Function 先保存身份，再创建独占对象
 5.2 transport 和 agent 的内部创建
 5.3 ops、FSM、protocol、backend 何时真正出现
 5.4 dataplane 与 queues：wrapper 存在不等于 ring 存在
 5.5 本组章节函数导航
6 从冻结资源到本 Function 的队列身份
 6.1 先固定本章的运行实例
 6.2 谁决定 BDF、BAR 和 qpair
 6.3 资源配置与驱动行为配置不是同一张表
7 Transport 接通以后，怎样发现设备并完成协商
 7.1 接通对象不是完成寄存器初始化
 7.2 从 Fabric BAR 租约到 capability 中的实际窗口
 7.3 status、feature mask 和失败返回
 7.4 与 full_init 的对照：不是同一次调用
8 建立 ring：对象、Host GPA 与设备 IOVA
 8.1 setup_queue 才是 ring 的分配入口
 8.2 split 256 的三块布局
 8.3 GPA 与 IOVA 分开追踪
 8.4 packed 与 DPU 私有 ring 的边界
9 把队列发布给设备：寄存器、通知、中断与 RX 预填
 9.1 队列 enable 是发布边界
 9.2 MSI-X 表、queue vector 与 notify 是三件事
 9.3 DRIVER_OK 之后谁预填 RX
 9.4 关键函数导航
10. 具体业务：一个 16 字节 TX 如何提交
 10.1 先看测试真正调用了什么
 10.2 submit_tx 的对象与失败契约
 10.3 header 和 payload 的字节从哪里来
 10.4 每一次提交的 DMA 所有权怎么建立
 10.5 两个 SG 如何变成 Split descriptor 与 avail
 10.6 真正的通知由谁发，submit 返回又意味着什么

11. 独立设备怎样消费 TX 并产生完成
 11.1 notify 从 PCIe 到设备 worker
 11.2 设备看到的 ring 地址怎样访问同一份 Host 内存
 11.3 从 avail.head 到 payload 字节
 11.4 一条链如何写成一个 used entry
 11.5 驱动读取 used，然后分两层回收
 11.6 monitor、scoreboard 与测试各证明什么
12. 具体业务：四帧 RX 从网络入口回到 packet_item
 12.1 这是独立测试，网络侧字节必须另有来源
 12.2 inject 只入设备 backlog，不直接填 Host buffer
 12.3 refill 发布四个可写 buffer，容量与有效长度分开算
 12.4 设备写 header、payload 和 used，used_len 说明实际写入多少
 12.5 Host 按 used 长度复制字节，再退休每份 DMA
 12.6 从 frame 到 packet_item，逐字节比较才闭合 RX 内容证据
 12.7 dataplane engines 是另一层调用入口，不是第二个 DMA 所有者
13 把各层连成一次完整运行
 13.1 从 run_test 到 flow ready
 13.2 golden TX 的顺序与证据
 13.3 独立 RX run：先到帧，后有可写 buffer
14 生命周期、故障定位和运行入口
 14.1 reset 的第一件事是证明设备停下
 14.2 按最早断开的边界定位
 14.3 MODEL 与 REAL_DUT 的替换边界
 14.4 如何复现，以及本文没有宣称什么
附录 A 按阅读目的查文件
附录 B 术语与阅读自检
 B.1 本文术语
 B.2 能回答这些问题，再继续改业务

1. 阅读范围与方法

1.1 本文从哪里开始，到哪里结束

本文讲解 `review/virtio-sdd-optimized` 分支，代码基线为 `e2a0a46cf968cb0cb0c3ad02a211ba65afc88d6c`。正文不从几十个类的 API 列表开始，而是从 UVM 创建 `virtio_net_env` 的上层 fixture 开始，逐个追问：这个句柄何时变成对象？内部还创建什么？谁把配置、内存和 PCIe 引用交给它？哪个函数真正分配 ring？哪一步设备才知道队列地址？一次真实提交又如何走到完成和资源释放？

主要代码入口是 virtio_real_driver_flow_test.run_phase():40 和它持有的 `virtio_real_driver_flow_fixture`。这个正式测试会经生产驱动接口提交一个 16 字节 TX payload，由独立 MODEL 设备消费描述符并写回 used ring；它不由测试直接填写描述符或伪造完成。

RX 则单独选择 virtio_real_driver_rx_test.run_phase():28，展开四个随机 IPv4/UDP 报文从设备网络侧入包，到驱动读回实际 Host 内存并做字节比较的过程。TX 和 RX 是两个独立测试运行，不把它们拼成源代码中并不存在的同一次测试。

所有本仓库源码链接都固定到上述完整提交，行号对应该版本。本文是源码教学和静态路径复核，不是新增的仿真通过报告；MODEL 中的“设备”也是 SystemVerilog 行为组件，不等于已经接入真实 RTL。

> **提示：**建议先读第 2～5 章把对象和 UVM 阶段弄清楚，再读第 6～9 章掌握地址、初始化和队列建立，第 10～12 章跟一次具体业务，最后用第 13 章的时间线复述完整过程。附录和各章的函数导航表用于回到源码。

1.2 先区分六种动作

动作	真正发生的事	尚未保证的事
声明 class 句柄	有一个保存引用的变量	不会自动创建对象
`new / type_id::create`	对象及其构造默认状态出现	不表示 PCIe 已绑定或设备已初始化
消费冻结 snapshot	得到 Host、Function、BDF、BAR、qpair 的权威视图	不等于分配 ring 或启动流量
分配 Host backing	manager 产生可访问的 GPA 字节存储	不自动建立 IOVA 访问权限
IOMMU 映射	为 `{host_id,BDF}` 建立 IOVA→GPA 和方向契约	不自动发布 descriptor 或通知设备

动作	真正发生的事	尚未保证的事
编程队列与发布 avail	设备知道 ring 地址，随后知道新 buffer 可用	不表示设备已写 used 或驱动已回收

例如 `transport` 的对象创建、它保存 frozen PCIe identity、它读到 capability、驱动把 queue_enable 写成 1，是四个不同的时点。`vq_mgr` 的对象也不是一组已经分配完的 ring；队列对象通常由后续 `ops.setup_queue()` 创建。

本项目没有 RDMA 中的 PD、MR、RC QP 状态机。Virtio 的 Function 相当于一个独立设备驱动上下文；VIO qpair 是控制面绑定的一对 RX/TX 资源，不是 RDMA QP；payload 和 ring 的 DMA 访问由 Host memory backing 与 IOMMU 映射承担，而不是 STAG/lkey/rkey 注册。

1.3 两条路径不能混讲

维度	本文具体测试主线	另外存在的路径
执行对象	MODEL 独立 `vdev_virtio_pci`	REAL_DUT 外部 RTL，经双向 PCIe 链路
初始化入口	fixture 手动调用生产 atomic ops	`fsm.full_init()` 批量初始化
业务入口	`flow.submit_tx/refill_rx/receive_rx`	UVM transaction 或 `function.dataplane`
队列格式	Split	Packed、DPU 私有 custom
Function 数量	Host0 的一个 VIO PF	snapshot 中多个 PF/VF/Host

最容易误读的地方是配置里 `DRV_MODE_AUTO` 的名字。driver 的 run_phase 等待 sequencer 请求，并不会仅因为这个默认枚举就自己执行 full_init。golden fixture 也明确不调用 full_init：先逐步进行 discovery、reset、feature 和 MSI-X 等动作，再由测试显式 setup 它需要的单个队列。本文会说明自动 FSM 如何调用相同能力，但不把对照路径说成当前测试的实际调用。

另一个误区是 `default_num_pairs=2`。它说明配置和 snapshot 为两对 qpair 预留能力，不代表这个 TX 测试已经创建了 RX0、TX1、RX2、TX3 的全部 ring。实际只建立测试所需的 TX 队列；RX 测试另建自己的 RX 队列。协商 MQ 后，软件预建拓扑还必须与设备接受的 active_pairs 区分。

1.4 默认主线参数与地址记法

参数来自 fixture.build_phase():99，不是为了讲课另造的 fixture。

参数	主线实际值或来源
VIO Function	Host0 / PF0；`vio_pf_count=1`
BDF、BAR 地址	从 frozen snapshot 解析，不硬编码
qpair 资源	每 PF 预留两对；qid 从 resource client 查询

参数	主线实际值或来源
queue_size / 格式	256 / VQ_SPLIT
device features	0x0000000100000001，CSUM 与 VERSION_1
driver feature mask	全 1，协商结果仍受 device features 限制
Host memory 范围	0x0000000100000000～0x00000001FFFFFFFF 配置区间
Host 分配策略	HOST_MEM_RANDOM；fixture 创建 Buddy manager
IOMMU	strict 开启，IOVA 默认随机策略
IRQ / NAPI budget	IRQ_MSIX_PER_QUEUE / 64
检查组件	scoreboard 开；coverage 关

随机分配意味着不能把一次运行的具体 GPA/IOVA 写成所有 seed 下的默认地址。后文用 G_desc/G_avail/G_used 表示三段 ring 的 GPA，用 I_desc/I_avail/I_used 表示设备可见的 IOVA；G_hdr/G_pkt、I_hdr/I_pkt 表示一次 TX 的 header/payload 地址。相同数值地址的合法复用也必须带上 Host 和 BDF 来讨论。

默认普通 Virtio-net 布局中 pair0 的 RX/TX qid 为 0/1，但测试仍调用 rx_queue_id_for_pair/tx_queue_id_for_pair 查询冻结的资源绑定。后文使用 qid=1 讲 TX 时，是该默认布局的结果，不是要求业务代码自行用 2*pair+1 重建权威 placement。

1.5 目录和层次分工

以下源码目录均位于 virtio_net_vip/src/，测试位于 virtio_net_vip/tests/。

目录	负责什么	不应让它代替谁
env/	env 装配、行为配置、执行模式、检查与附件	不重分配 DPU 权威身份
sriov/	PF/Function/VF 包装与资源 client	不私造 BDF/BAR/qpair
agent/	driver、ops、FSM、monitor、DMA 所有权	不代替设备完成 DMA
transport/	PCI 配置/BAR/capability/notify 与观察适配	不另建 Host allocator
virtqueue/	ring 格式、descriptor、avail/used、回收	不自行代表真实通知已发送
iommu/	Host-qualified IOVA 翻译、方向权限和故障	不分配 BAR 或 payload

目录	负责什么	不应让它代替谁
`dataplane/`	TX/RX/offload 高层业务适配	生产 buffer/notify 委托 ops
`devmodel/`	MODEL 独立设备解析、DMA 与完成	不在 REAL_DUT 下偷偷工作
`pcie/`	设备 DMA 和 REAL_DUT Host 服务适配	不接管外部 PCIe 生命周期

`dpu_common`、`host_mem`、`pcie_work`、`net_packet`、`queue_work` 是外部依赖。教学文档只消费本项目暴露的契约，不把它们复制进 Virtio 仓库，也不修改其实现。

2 从正式测试入口认识对象树

2.1 阅读基准与第一条调用链

本章到第 5 章以提交 `e2a0a46cf968cb0cb0c3ad02a211ba65afc88d6c` 为准，路径均相对仓库根。主线是 `virtio_net_vip/tests/virtio_real_driver_flow_test.sv` 和同目录的 `virtio_real_driver_flow_fixture.sv`，默认执行模式 MODEL。这里的“真实驱动流程”表示调用生产 transport/atomic ops 并经过 PCIe 设备模型完成业务，不表示连接真实 RTL。

测试的 build_phase（第 36 行） 只有一个关键动作：创建 `flow` fixture。配置、外部子环境和 MODEL 附件由 fixture 的 build_phase（第 99 行） 继续创建。`type_id::create` 先构造对象；UVM 随后的 build 遍历才调用子组件的 `build_phase`，不能把两件事视为同一时刻。

默认一个 PF 的组件层次如下。方括号表示普通对象或借用句柄，不一定出现在 UVM component 拓扑打印中。

```
uvm_test_top : virtio_real_driver_flow_test
└─ flow : virtio_real_driver_flow_fixture
   ├─ [device_builder / device_cfg / virtio_cfg / pcie_cfg]
   ├─ [host_mem_owners : host_mem_pool → Host 0 manager]
   ├─ device_env : dpu_device_env
   │  └─ virtio_env : virtio_net_env
   │     ├─ pf_0_0 : virtio_pf_instance
   │     │  ├─ [pf_manager / lifecycle_reset_owner]
   │     │  └─ pf_function : virtio_function_instance
   │     │     ├─ driver_agent → driver / sequencer / monitor / observer
   │     │     └─ [transport / vq_mgr / dataplane / resource_client]
   │     ├─ perf_mon
   │     ├─ scb（默认创建）; cov（默认关闭）
   │     └─ [iommu / wait_pol / barrier / err_inj / conc_ctrl / dyn_reconfig]
   ├─ pcie_env : pcie_tl_env（RC/EP agent）
   ├─ vdev0 : vdev_virtio_pci
   └─ [tlm_adapter / device_port / m_profile]
```

`pcie_env` 是 fixture 的子组件，与 `device_env` 同级，不是 `virtio_env` 创建或拥有的子组件。设备模型 `vdev0` 也在 fixture 下；Virtio 驱动和设备模型不能因名字相近就归为同一个对象。env 的 `function_instances[]`、`vf_instances[]` 是访问视图，其中句柄指向 PF 容器已经创建的 Function，不是另建一份 Function。

2.2 fixture 先准备什么，为什么必须在 env 之前

fixture 构造函数（第 81 行）把就绪位清零，设置 `vio_pf_count=1`、PCIe completion 超时 100000 ns、轮询超时 1000ns，以及 `device_features=64'h0000_0001_0000_0001`，即默认 CSUM 与 VERSION_1。构造期没有 ring、DMA 映射或 PCIe 请求。

build 首先创建 `virtio_test_device_builder`，声明 Host 0/domain、PF、三类 BAR 和 VIO service，并提交总数为 `2 * vio_pf_count` 的固定 qpair 请求；第一个 PF 被选为 AF。`vio_pf_count` 为 0 或超过 DUT 能力时立即 fatal。这里写的是控制面输入，最终 BDF、BAR 和资源绑定必须等 DPU 发布快照后读取；fixture 没有用常量代替快照里的 BDF。

第 133 行创建 `virtio_cfg`。正式主线把队列对数设为 2、ring 大小 256、格式 `VQ_SPLIT`、driver 支持特性全 1、RX 模式 MERGEABLE、每队列 MSI-X、NAPI 预算 64，开启 scoreboard、关闭 coverage。Host GPA 范围与 IOVA 范围是两套不同地址域：前者交给共享 Host allocator，后者在 env 内配置 IOMMU。之后 `apply_plusargs(why)` 解析模式覆盖，失败 fatal；`execution_profile()` 返回模式策略，非法编码可能使其为 null，也会阻止继续构建。

第 189 行创建 `host_mem_owners`，调用 `create_host(host_id, mem_base, mem_end, MODE_BUDDY, DEFAULT_MIN_GRANULE, policy)` 建立唯一 Host backing。失败 fatal。相同池随后同时交给 `device_cfg.host_mem_pool_ref` 和 `virtio_cfg.host_mem_pool_binding`；PCIe 配置通过 `bind_host_memory(0, host_id, root_mem, why)` 借用池中的同一个 manager，并发布 `dev_mem_0`。这解释了之后驱动写 ring、设备 DMA 读 ring 为何能够看见相同字节，而无需在几个环境之间复制内存。

实际创建次序的短节选如下；它展示父对象和配置路径之间的一一对应关系。

```
device_env = dpu_device_env::type_id::create("device_env", this);
uvm_config_db#(virtio_net_env_config)::set(
    this, "device_env.virtio_env", "cfg", virtio_cfg);
m_virtio_env = virtio_net_env::type_id::create(
    "virtio_env", device_env);
```

出处：fixture 第 203～207 行。配置发布目标是 `device_env.virtio_env`，不是 `flow.virtio_env`；排查“No virtio_net_env_config”时先核对完整层次。

2.3 execution profile 与 MODEL 附件

策略定义见 virtio_execution_profile.sv。`virtio_net_env_config.execution_profile()`（配置文件第 256 行）委托 `virtio_select_execution_profile`，MODEL 与 REAL_DUT 各创建一种无状态策略对象。fixture 把结果保存在 `m_profile`。因此 execution profile 不属于 `virtio_net_env` 的成员；是配置接口生成、fixture 保存并用于选择附件的对象。

选择结果	MODEL 主线的实际作用	REAL_DUT 对照边界
`has_model_device()`	创建 completion adapter、EP 设备 port 及每 PF 一个 vdev	不创建设备模型
`pcie_if_mode()`	选择 TLM 连接模式	选择 SV 接口模式

选择结果	MODEL 主线的实际作用	REAL_DUT 对照边界
`ep_is_active()` / `ep_auto_response()`	EP active、自动应答配置启用	EP passive、禁用自动应答
`needs_physical_link()`	不要求物理 VIF	要求独立 RC→EP 和 EP→RC 接口
`supports_host_mem_responder()`	不走 REAL_DUT responder 分支	允许可选 RC Host-memory 服务

MODEL 分支在创建 PCIe env 前先调用 `tlm_adapter.install_factory_overrides()`，再把 `pcie_env.*` 中的 `pcie_tl_ep_driver` 覆盖为 `vdev_ep_driver`（fixture 第 212～217 行）。顺序重要：若已经创建 driver 再安装覆盖，现有对象不会自动变成新类型。第 253 行创建 PCIe env，第 256 行创建普通对象 `device_port`，随后创建 `devices[]` 里的组件。`host_mem_responder` 是 REAL_DUT 分支对象，MODEL 默认为空；不能把它写成 MODEL 完成 DMA 的核心。

连接时 `device_port.connect_ep(ep_driver, ep_monitor, iommu, host_id, why)` 借用 PCIe EP 驱动、监控及 Virtio 环境 IOMMU；设备初始化留到 `end_of_elaboration_phase`。后者取得冻结快照、AF 身份、三个 BAR 和共享内存，再调用 `configure_devices(why)`（第 1124 行）：逐 Function 检查 PCIe 身份及 BAR 租约容量，设置 vdev 的 BDF、最大 pair 数、queue 上限、特性和错误注入引用，执行 `init()`，最后 `device_port.attach(devices[k])`。port/env/snapshot 为空、模型数量不足、BAR 租约过小或没有有效 Function 都会失败。这个连接把驱动一侧的 IOVA 翻译与设备一侧的 DMA 接到相同上下文。

3 virtio_net_env 的每个成员从哪里来

3.1 配置与权威快照：借用外部，创建本地只读上下文

env 定义及完整成员从 virtio_net_env.sv 第 40 行 开始。`new(name,parent)`（第 87 行）仅初始化 `configuration_valid=0`、`effective_dut_caps=null`、`snapshot_ctx=null` 和协议接口索引 0。名称与父组件用于 UVM 层次注册，不构造业务组件。

成员	谁创建或发布	env 做什么／所有权
`cfg`	fixture 创建，通过 config_db 发布	build 取得同一配置句柄；借用
`device_snapshot`、`resource_snapshot`	DPU 控制面发布	借用冻结对象，不重分配 BDF/BAR/qpair
`device_resource_manager`	DPU 控制面播种并发布	借用；须对应同一快照对
`snapshot_ctx`	env 调用 `vio_snapshot_ctx::create_checked`	本地创建包装对象；内部保存三者引用

成员	谁创建或发布	env 做什么／所有权
`effective_dut_caps`	`device_snapshot.snapshot_dut_caps()`返回	保存能力副本；对外再复制
`configuration_valid`	env 内部状态	build/绑定失败的后续 phase 守卫，不是设备 ready
`protocol_event_vif_index`	env 内部计数	成功提交绑定后记录已用协议接口槽数

`build_phase(phase)`（第 255 行）由 UVM 调用，先 get cfg、验证执行模式，然后读取三个 DPU 句柄。`vio_snapshot_ctx::create_checked(device_snapshot, resource_snapshot, manager, why)` 位于 vio_snapshot_ctx.sv 第 23 行：device 快照非空且冻结，resource 快照非空、冻结且引用该 device 快照，manager 非空、已播种且来自同一对快照，全部成立才创建上下文；任一不符返回 null 和 why。env 将 null 转为 fatal 并返回。

这不是仅比较版本号。两份内容看似相同但并非发布时配对的快照也不能随意替换。随后 `cfg.validate_against_snapshot(device_snapshot,why)` 检查行为配置和权威拓扑关系，读取 DUT 能力。`snapshot_effective_dut_caps()`（第 235 行）在未取得能力时返回 null，否则新建副本并 copy，调用方修改返回对象不会改动 env 持有的能力。

3.2 Host memory、IOMMU 与共享策略

env build 第 304～397 行决定共享基础设施。`host_mem` 有三条合法来源，只有第三条在 env 内创建 allocator。正式 fixture 走第二条。

分支	输入与结果	真实拒绝条件
注入 manager	`cfg.host_mem_binding` 直接赋给 `host_mem`	manager 的 Host ID 与 cfg 不一致
注入 pool	从 `cfg.host_mem_pool_binding` 或 `dpu_host_mem_pool` 取池，再 `get_host(cfg.host_id)`	类型错误、两处池不是同对象、Host 缺失或返回 manager 不匹配
无注入	创建 `host_mem_manager`，设置 Host、分配策略、GPA 范围	后续 BAR 导入等校验仍必须通过

直接 manager 和 pool 不能同时提供。BAR importer 是 `build_phase` 局部对象 `bar_importer`，不在 env 成员表里；其 `import_snapshot(device_snapshot,host_mem,host_id,why)` 在任何 ring 分配前预留与 GPA 范围相交的 BAR，重复导入允许幂等。导入失败使配置无效并 fatal。

`iommu` 始终由此 env 创建，设置 `strict_permission_check` 并调用 `configure_iova_aperture(iova_base,iova_limit,iova_alloc_policy,why)`。失败意味着 IOVA 配置不可用，不应继续建队列。要特别区分：共享 Host 内存池的 manager 由外部拥有，并不自动意味着每个业务 env 也共享同一 IOMMU 对象。此源码中 IOMMU 是在每个 env 创建，再借给其 Function 和 MODEL 设备端口。

`wait_pol`、`barrier`、`err_inj` 是 env 创建的普通对象，之后同一批引用注入 Function、queue manager 和 ops。等待策略管超时；barrier 表达 ring 发布/读取顺序并统计；err_inj 提供故障注入上下文。它们不是每条队列单独创建。transport 构造时自己的默认 wait policy 会在提交绑定时被这个共享 `wait_pol` 替换。

3.3 PF、Function、VF 数组并不是三套设备

`build_snapshot_topology(output why)`（env 第 98 行）由 build 调用。先按完整 service key 得到 VIO 服务集合：空白名单表示全部 VIO service；非空白名单精确选择。若启用 Host scope，必须先向快照解析 service owner，再根据 `owner.host_id` 筛选，不能按数组下标推断 Host。

全局无 VIO service 且没有 scope 时返回失败；有显式 scope 而筛选为空时允许空 PF/Function/VF 数组成功，保留共享基础设施。每个选中 service 必须有 resource snapshot 中的 qpair binding。然后把 owner 转换成父 PF key，按 Host/PF 归组，创建 `pf_instances[]` 并调用每个 PF 的 `configure_services_ctx(parent_key,snapshot_ctx,grouped_services,why)`。

PF 容器再创建 PF Function 和 VF Function。env 两次调用 `collect_functions(ref queue)`：第一次计算总数，第二次把已有句柄放入 `function_instances[]`；最后筛选 `function_kind==DPU_FUNCTION_VF` 并 `$cast` 到 `vf_instances[]`。cast 失败报错并返回 0。默认 fixture 只声明 PF，故一个 PF 容器、一个 PF Function、VF 数组为空是正常结果。

`pf_mgr` 是兼容别名：env connect 中赋值为 `pf_instances[0].pf_manager`，不是 env 另建的全局 PF 控制器。每个 PF 容器原本各有自己的 manager；阅读多 PF 代码时不可把 `env.pf_mgr` 当成覆盖所有 PF 的权威对象。

3.4 验证、性能、并发与动态重配置成员

成员	创建位置	连接及主线使用边界
`perf_mon`	env build 第 390 行，无条件创建组件	复制带宽开关和 Mbps；自身 build 初始化 token bucket；本 env 未把它自动接到每个 TX 调用
`scb`	env build，`cfg.scb_enable` 为 1 才创建	每 Function monitor 的 `txn_ap` 接 `scb.txn_imp`
`cov`	env build，`cfg.cov_enable` 为 1 才创建	`txn_ap` 接 `cov.analysis_imp`；golden 默认不创建
`conc_ctrl`	env build 创建普通对象	connect 借用 VF 数组和 wait policy；默认无 VF，golden 不发并发调度
`dyn_reconfig`	env build 创建普通对象	`bind_device_snapshot(device_snapshot,why)`绑定冻结拓扑，失败 fatal；golden 不调用动态重配置

成员	创建位置	连接及主线使用边界
`pcie_env_ref`	env 只声明 `uvm_object` 字段	本文件正式流程没有给它赋值；实际 PCIe 借用从 `bind_pcie` 参数进入

scoreboard 的 `new`（`src/env/virtio_scoreboard.sv:57`）清统计，`build_phase:67` 创建 `txn_imp`；coverage 的 `new`（`src/env/virtio_coverage.sv:247`）创建 8 类 covergroup，`build_phase:263` 创建 analysis 入口。perf 的 `build_phase`（`src/env/virtio_perf_monitor.sv:55`）在限速开启且速率大于 0 时，以 `Mbps*125` 初始化 1ms 容量的 bucket。这些对象“已经创建”并不等价于 golden TX 必然触发所有高级校验或性能采样；应继续追真实调用和 analysis 事件。

4 UVM phase 如何把对象变成可执行流程

4.1 build：先有拓扑身份，再有 Function 内部对象

UVM build 是父向子展开。fixture 先创建 DPU 父环境和 Virtio 子环境；Virtio env 消费已发布快照，创建 PF 容器并立即配置 service；PF 配置又创建 Function、写入身份。之后 UVM 才进入 Function 的 `build_phase`，创建 transport/queue manager 等。因此 `configure_from_ctx()` 第一次执行时内部子对象可以尚未存在，这正是 `apply_function_configuration()` 有空句柄早返回，并在 Function build 尾再调用一次的原因。

PF 容器的 configure_services_ctx（第 98 行） 接收父 PF key、上下文和 service 队列，先校验：只能配置一次，父 key 必须 PF 且 vf_id=0，manager 有 `virtio.qpair`，manager 与 snapshot 能力一致，服务非空且为 VIO，owner 属于同一 PF 组且不重复，PCIe 身份、三个 BAR 角色、资源 manager 中的 Function 都能解析。通过后才创建 manager 和 PF/VF 子组件、导入资源池视图。失败返回 0 和 why；全部前置检查前不创建子组件，但后续创建/导入阶段失败不是删除已经创建的 UVM 组件。

PF 的 `build_phase:318` 要求已经配置 service，给 manager 设 pf_index；`connect_phase:334` 把本 PF 的 VF 数组、PF transport 交给 manager，创建 `lifecycle_reset_owner`，使 Admin-VQ 全复位委托到普通 PF Function 的生命周期。这个 owner 只是复位路由桥，不拥有 ring 资源。

4.2 connect：先装行为和 analysis，再由 fixture 接 PCIe

env 的 connect_phase（第 433 行） 在配置无效时直接返回。正常情况为每个 Function 列出该 service 的 qpair bindings，以数量作为 `allocated_qpairs`，调用 `cfg.get_service_config(service_key,allocated_qpairs,driver_cfg,why)`，把结果赋给 `function.drv_cfg`。配置对象内部有显式 service 覆盖时取覆盖，否则取默认；请求 pair 数会收缩到分配上限，`max_vio_net_qpairs_per_device` 同步为该上限。binding 为 0 或配置解析失败都会停止连接。

随后按 service 取 `private_cfg`，连接 monitor analysis 端口，给 `conc_ctrl` 与兼容 PF manager 传 VF 句柄。这里没有寄存器访问，也没有 ring 创建。标准 golden 没有 private 配置，所以之后选标准协议。

UVM connect 为子向父执行。agent 自己的 connect 可能早于 fixture 最终 PCIe 绑定，此时 ops/FSM 尚未创建并不奇怪。Function 提交绑定会直接更新 driver 引用，agent 还在 `start_of_simulation_phase` 再执行 `apply_component_bindings()`，并检查 active driver 是否已有 ops/FSM。agent 文件头“before build 注入”的旧概括不足以描述这条正式路径，应按函数体和 UVM 遍历次序理解。

fixture 的 `connect_phase:263` 检查 PCIe RC/EP agent、monitor、RC sequencer 后调用：

```
m_virtio_env.bind_pcie(
    m_pcie_env.rc_agent.sequencer, tlm_adapter,
    m_pcie_env.rc_agent.monitor, m_pcie_env.ep_agent.monitor)
```

成功后仅置 `m_pcie_path_bound=1`，设备还未初始化。MODEL 设备 port 随后接 EP 驱动和 IOMMU；至 end_of_elaboration 完成设备配置后才置 `m_endpoint_ready=1`。`build_flow(output why)`（fixture 第 412 行）是测试 run 调用的就绪检查函数，检查 endpoint、冻结快照、PCIe 通路和共享句柄，成功置 `m_flow_ready`。名字含 build 不表示它再次创建环境。

4.3 PCIe 绑定的预检、提交与失败边界

bind_pcie（env 第 514 行） 接受一个必需 RC sequencer、可选 completion adapter 和两侧 monitor。它把相同外部通路加入每 Function 的绑定计划；只能描述同一个 PCIe domain，不能把一条 sequencer 广播给多个独立 domain。空 sequencer、身份未解析、跨 domain 或 monitor 的 analysis port 为空都会失败。多 domain 应调用 `bind_pcie_endpoints(endpoints)`，按完整 `{host,segment,BDF}` 选择，要求一个 Function 恰好一个有效 endpoint，无重复身份、无多余 endpoint，且 transport 身份匹配。

两入口都进入 `bind_all(plan,legacy_adapter)`（第 704 行）。应按下表的副作用边界阅读。

内部函数	输入／动作	失败含义
`preflight_plan:754`	每 Function 预检 observer、协议 VIF、ops/FSM 候选	拒绝两个 Function 共享同一 VIF/FSM/ops 运行态
`resolve_protocol_vif:894`	从 `uvm_test_top` 取 `protocol_event_vif_<slot>`，或复用暂存 VIF	槽号达到 DPU_MAX_FUNCTIONS、配置缺失/null 则 fatal
`preflight_function_pcie:923`	observer 强制绑定预检，再 Function 预检；成功推进暂存槽号	缺 driver agent/monitor/observer 或预检失败返回 0
`validate_plan_adapters:799`	只读检查 adapter/domain/RC driver 关系	同 driver 喂不同 adapter、同 domain 对应不同 driver、同 driver 跨 domain 均拒绝
`commit_plan_adapters:849`	每个 adapter/domain 组合绑定一次 RC driver	底层绑定失败返回 0

内部函数	输入／动作	失败含义
`bind_function_pcie:954`	注入共享基础设施，提交 observer，装协议 VIF，连接两侧 monitor	Function 提交失败则当前绑定失败
`abort_bind:743`	清计划内 Function 的 pending 候选，置 env 无效	返回 0；不是析构或复位整套已发布运行状态

重要边界：源码注释强调“原子”绑定；函数体保证全部 Function 预检通过前不逐个提交，并在失败时清候选。但 `abort_bind` 实现只有 `cancel_bind_pcie()` 和 `configuration_valid=0`，没有遍历断开已连接的 analysis 端口或撤销已提交的 adapter。因此不能把它夸大成任意提交中途失败都能完全恢复历史状态的通用事务系统。文档按可观察实现说明，而不沿用超出实现的保证。

`bind_function_pcie` 最后把 RC、EP monitor 的 `tlp_ap` 接到本 Function observer 的 analysis 入口；若两个 monitor 是同对象只接一次。observer 完成身份过滤和语义解码，monitor 再发布 Virtio 事务。协议 VIF 槽由 testbench 预先创建，env 只借用；如果顶部没有发布该 VIF，即使 MODEL PCIe 本身不需要物理 VIF，协议检查连接仍会失败。

4.4 run 与 report：env 没有自动初始化线程

本基线的 `virtio_net_env` 没有 `run_phase` override；`virtio_driver.run_phase`（`src/agent/virtio_driver.sv:50`）永远等待 sequencer 事务，取到后处理并 `item_done()`。`VIO_TXN_INIT` 才会分派 `fsm.full_init()`。默认配置 `DRV_MODE_AUTO` 不能推导“run 一开始自动 full_init”。

正式 golden 测试的 run 先 raise objection，再 `build_flow`、检查快照、通过 resource client 解析 TX queue ID，随后 `start_driver_flow`、`setup_queue`、TX submit/complete、reset teardown，最后 drop objection。fixture 的 `start_driver_flow:736` 明确逐步调用生产 atomic ops 完成初始化，不调用 full_init；这样 ring 只在 `setup_queue` 创建一次。该基线也没有 fixture 先设置 `DRIVER_MANUAL` 的代码。后续业务章节将沿这条实际调用链展开，FSM 全自动初始化属于另一入口。

env 的 `report_phase:997` 在配置无效时跳过；正常调用 Host-memory、IOMMU 和每 Function queue manager 的 `leak_check()`，再打印 barrier 统计。它检查但不替代 reset teardown 释放资源。scoreboard、coverage、perf 自己的 report 由 UVM 分别调用。因而“测试结束会自动 report”不能作为未完成设备复位也可安全释放 DMA 的理由。

5 Function 内部：创建、身份应用、协议选择与延迟建队列

5.1 Function 先保存身份，再创建独占对象

virtio_function_instance.sv 第 16～54 行区分三类成员。身份值包括 function/service key、完整 PCIe ID、BDF、kind、vf_index 和 BAR 数组；外部引用包括 resource manager、snapshot 上下文、pcie_ctx_ref 和共享 mem/IOMMU/barrier/err_inj/wait_pol；本 Function 构造的对象包括 driver agent、vq_mgr、dataplane、transport、resource client，以及绑定阶段选出的 protocol/ops/FSM/backend。`pcie_ctx_ref` 可选、由外部 PCIe 管理方拥有，当前 PF 创建路径未传它，默认 null。

`configure_from_ctx(ctx,service_key,pcie_ctx=null)`（第 150 行）由 PF 容器调用。它根据 service 解析 owner、PCIe ID 和 DEVICE_MEMORY/MAILBOX/MSIX 三种 BAR，检查 manager 确实包含 Function，禁止已经绑定的 Function 改挂另一 device/resource 快照、service、manager 或 PCIe context。若 resource client 已经创建，先尝试绑定资源，再发布成员；成功返回 1 并调用 `apply_function_configuration()`。缺上下文、非 VIO key、身份/BAR 缺失或归属重绑都返回 0 并保留 fatal 诊断。

`build_phase:106` 的核心仅五次 create：

```
driver_agent = virtio_driver_agent::type_id::create("driver_agent", this);
vq_mgr = virtqueue_manager::type_id::create("vq_mgr");
dataplane = virtio_net_dataplane::type_id::create("dataplane");
transport = virtio_pci_transport::type_id::create("transport");
resource_client = virtio_resource_client::type_id::create("resource_client");
apply_function_configuration();
```

`apply_function_configuration:588` 要求 transport/vq_mgr/resource_client 存在且 PCIe 身份有效，否则早返回；正常把完整 ID 交给 transport，把 Host/BDF 交给 queue manager 和数据面引擎，把 BAR 租约交给 bar accessor。若 resource client 未绑定 service，则绑定同一个 snapshot_ctx/service，失败 fatal；最后 `transport.configure_fabric_managed(resource_client)` 标明由 Fabric 资源视图管理。这里仍没有 Host ring 分配。

5.2 transport 和 agent 的内部创建

transport 的 new（第 79 行） 创建 `cap_mgr`、`bar`、`notify_mgr`、默认 `wait_pol` 和 `virtio_fabric_preconfigured_discovery`。cap manager 和 notify manager 指向同一个 bar accessor；初始 queue 数、特性为 0，BDF 为 0，身份 valid 为 0。bar 上的 RC sequencer 在之后提交绑定才注入，所以“transport 已构造”不代表能访问设备。

agent 的 build_phase（第 55 行） 总会创建 monitor 和 observer，只有 `UVM_ACTIVE` 才创建 driver、sequencer。monitor 保存解码后的事件及协议检查接口；observer 接 PCIe TLP 再向 monitor 解释。agent 并不在此函数创建 ops/FSM，它们只是待注入成员。

`connect_phase` 把 active driver 的 seq_item_port 接 sequencer 导出，并调用 `apply_component_bindings()`：非空 ops/FSM/protocol 传给 driver，monitor 缺少 transport/vq_mgr 时从 ops 补齐，复制协商特性视图，observer.monitor 指向 monitor。这个函数不分配资源、不启动事务。`start_of_simulation_phase` 再次调用它；active driver 缺 ops/FSM 报 UVM_ERROR。这样可以处理父 fixture 晚于子 agent 执行 connect 的情况。

5.3 ops、FSM、protocol、backend 何时真正出现

Function 的 `preflight_bind_pcie(pcie_rc_seqr)`（第 252 行）首先清上一次 pending，检查 sequencer、agent、transport、vq_mgr 及 transport 三个必需子对象。它从 agent 已有 ops/FSM 取候选，再调用 `prepare_driver_candidates(label,drv_cfg,ref ops,ref fsm)`（第 73 行）。标准主线 `private_cfg==null`，创建 `virtio_standard_driver_protocol`；私有配置非空才创建私有协议。不是全局 factory 覆盖所有 Function 的驱动语义。

基类协议的 prepare_candidates（第 40 行） 仅在候选为 null 时创建通用 FSM 和 atomic ops，create 的 context 参数来自当前 Function。已有候选允许复用，但 env 跨 Function 别名检查禁止共享运行态。Function 拒绝 factory 返回 null，并检查 FSM 能绑定当前 qpair 上限；成功保存 pending 对象并置 `pcie_bind_prepared=1`，尚未发布正式共享引用。

`commit_bind_pcie(hmem,iommu_mdl,bar_mdl,einj,wpol,pcie_rc_seqr)`（第 323 行）要求预检成功且 pending ops/FSM 非空，否则 fatal 返回 0。它委托 `commit_validated_pcie_components`（第 373 行）进行如下装配。

接线目的	本次赋值或调用	为什么
配置后端	`select_config_backend(label)` 创建 common 或 private backend，再 `bind_transport`	协议和寄存器布局保持一致
协议提交	`pending_protocol.bind_ops(ops,backend,driver_cfg,why)`	标准要求 common backend；私有配置在此冻结
FSM 上限	`fsm.bind_mq_pair_limit(...)`	初始化不能越过资源分配的 pair 能力
queue manager	传 mem/IOMMU/barrier/err_inj/wait policy	每条队列用同一共享基础设施
transport	传 wait policy、RC sequencer；统一 bar 引用	配置访问/notify 走真实 PCIe 通路
ops	传 transport/vq_mgr/mem/IOMMU/wait policy	一个 ops 集中拥有正常队列 DMA 生命周期

接线目的	本次赋值或调用	为什么
dataplane	协议安装，写 Host/BDF，`bind_ops(ops)`	TX/RX engine 借用同一个 ops 而非另做分配
agent/driver/monitor	发布 ops/FSM/protocol 及 transport/vq_mgr	事务驱动、手工 ops 和监控使用同一上下文

标准协议 `bind_ops`（protocol 文件第 108 行）拒绝 null 后端以及不是 `virtio_common_cfg_backend` 的对象，成功设置 `ops.config_backend` 和 `ops.protocol`。Function 还会检查协议提交和 FSM 上限绑定的返回值；失败清 pending 返回 0。最终 Function 保存共享基础设施和正式 protocol，再清 pending 引用。`cancel_bind_pcie` 只是丢弃预检暂存，不会销毁已创建对象或释放 Host-memory。

5.4 dataplane 与 queues：wrapper 存在不等于 ring 存在

dataplane 的 new（第 39 行） 创建 TX engine、RX engine 和 offload engine。Function 提交时直接注入共同 offload、Host/BDF 和 ops，标准协议的 `install_dataplane` 为空实现，保留默认引擎。当前 Function 没有调用 dataplane 的 `configure(features,mtu,mss,rx_mode,rx_buf_sz,rx_refill_thresh,bdf,host_id)`；不能仅因该 API 存在就说正式初始化已同步所有 offload 参数。golden TX 直接调用 ops，也没有绕经这个 wrapper 的 TX engine。

`bind_ops(ops)`（dataplane 第 92 行）只把同一 handle 放进两个 engine；不检查 null、不分配 DMA，调用者必须在收发前准备好 ops。`configure` 若由其他上层调用，则同步特性、MTU/MSS、RX 缓存参数和 DMA 身份到 engine/offload，函数本身不返回成功标志。

queue manager 的构造函数（`src/virtqueue/virtqueue_manager.sv:45`）不建任何 queue。真正的 create_queue(queue_id,queue_size,vq_type)（第 63 行） 按类型创建 split/packed/custom 队列，调用 `vq.setup(...,mem,iommu,barrier,err_inj,wait_pol,bdf,host_id)`，再登记 `queues[queue_id]`。重复 ID 报错但返回已有对象，不是返回 null；未知类型才返回 null。它是对象建立入口，后续 atomic queue setup 负责完整 ring 分配/映射/寄存器编程及失败处理，不能把 manager 的一次 create 当成队列已经在设备启用。

因此到 PCIe connect 成功时，应该看到可用的资源 client、transport、driver agent、ops/FSM 和空 queue manager。到测试 `setup_queue` 成功后才有具体 queue 及 ring 资源。到 TX 完成后释放的是该报文的 DMA 资源；整条 queue 直到 verified reset/teardown 才退役。这个时间顺序是后续章节理解“谁申请、谁映射、谁通知、谁回收”的基础。

5.5 本组章节函数导航

下面位置均在固定 SHA 的 `virtio_net_vip/` 下；上文同文件 GitHub 链接可直接定位，行号是函数定义行或紧邻入口。表中列出本组装配主线的导航，不替代正文中的参数和失败解释。

文件与入口	调用者	阅读目的
`tests/virtio_real_driver_flow_test.sv:36 build_phase`	UVM	唯一 fixture 创建入口
`tests/virtio_real_driver_flow_fixture.sv:99 build_phase`	UVM	配置、共享池、DPU/Virtio/PCIe/MODEL 附件创建
同文件:`263 connect_phase`、`:307 end_of_elaboration_phase`	UVM	接外部 PCIe；解析快照、初始化设备
同文件:`412 build_flow`	test.run_phase	检查就绪，发布 flow_ready
同文件:`736 start_driver_flow`	test.run_phase	逐步生产 ops 初始化，不调用 full_init
同文件:`1124 configure_devices`	fixture.end_of_elaboration	快照身份/租约转为 MODEL 配置
`src/env/virtio_net_env_config.sv:197 make_default_driver_config`、`:339 get_service_config`	env.connect	从 service 资源上限派生行为
`src/env/virtio_net_env.sv:87 new`、`:255 build_phase`	factory/UVM	建基础设施和 snapshot 拓扑
同文件:`98 build_snapshot_topology`	env.build	PF 分组，Function/VF 只读访问视图
同文件:`235 snapshot_effective_dut_caps`	外部读者	返回能力防御性副本
同文件:`433 connect_phase`	UVM	行为和 analysis 接线
同文件:`514 bind_pcie`、`:580 bind_pcie_endpoints`	fixture/集成环境	单 domain 通路或按完整 ID 选择 endpoint
同文件:`704 bind_all`、`:743 abort_bind`	两个绑定入口	统一预检、提交、pending 清理
同文件:`754 preflight_plan`、`:799 validate_plan_adapters`	bind_all	跨 Function 运行态与 adapter 拓扑检查
同文件:`849 commit_plan_adapters`、`:954 bind_function_pcie`	bind_all	外部 PCIe/observer/协议 VIF 装配
同文件:`997 report_phase`	UVM	leak 检查和 barrier 统计
`src/sriov/vio_snapshot_ctx.sv:23 create_checked`	env/PF 独立入口	校验冻结对与 manager 同源

文件与入口	调用者	阅读目的
`src/sriov/virtio_pf_instance.sv:98 configure_services_ctx`	env.build_snapshot_topology	验证组并创建 PF/VF
同文件`:306 collect_functions`、`:318 build_phase`、`:334 connect_phase`	env/UVM	视图收集、manager、reset owner
`src/sriov/virtio_function_instance.sv:150 configure_from_ctx`、`:588 apply_function_configuration`	PF 配置/Function.build	身份、BAR、resource client 安装
同文件`:106 build_phase`	UVM	Function 五类独占对象创建
同文件`:73 prepare_driver_candidates`、`:252 preflight_bind_pcie`	env.preflight	选择协议、准备 ops/FSM 候选
同文件`:323 commit_bind_pcie`、`:373 commit_validated_pcie_components`	env.bind_function_pcie	发布共享依赖和正式驱动对象
同文件`:363 select_config_backend`	Function 提交内部	创建协议对应寄存器后端
`src/agent/virtio_driver_protocol.sv:40 prepare_candidates`	Function 预检	懒创建通用 ops/FSM
`src/agent/virtio_driver_agent.sv:55 build_phase`	UVM	active/passive 组件边界
`src/transport/virtio_pci_transport.sv:79 new`	Function.build	cap/bar/notify/default 策略创建
`src/dataplane/virtio_net_dataplane.sv:39 new`、`:92 bind_ops`	Function build/提交	engine 创建及共同 ops 注入
`src/virtqueue/virtqueue_manager.sv:63 create_queue`	atomic queue setup	延迟创建具体队列对象

6 从冻结资源到本 Function 的队列身份

6.1 先固定本章的运行实例

本章延续 `virtio_real_driver_flow_test` 的正式 MODEL 路径。测试创建 `flow`，在 run phase 先检查 `build_flow(why)`，从 placement 查询 pair 0 的 TX 队列号，调用 `start_driver_flow(ok)`，然后执行 `setup_queue(tx_queue_id, 256, VQ_SPLIT, ok)`。其后才提交一份 16 字节 payload。初始化阶段并没有构造一个 sequence transaction，也没有调用 `fsm.full_init()`。配置里的 `DRV_MODE_AUTO` 不是自动发生初始化的证据。

阅读入口：golden test，L46、fixture 初始化，L736。后面提到完整 FSM，是解释同一批底层接口的另一条调用路径，不能把两条路径拼成一次运行。

本次实例的参数	实际来源	本次含义
Host 0、一个 VIO PF	fixture 构造默认值与 builder 请求	首个 PF 被选为 AF；BDF 仍由冻结 snapshot 提供
每 PF 两对 qpair	builder 的固定 VIO 资源请求	资源已预留，不代表设备已接受两对 active queues
`num_queue_pairs=2`	fixture 默认行为，经 service 配置按分配数限制	用于容量检查与向量规划
`queue_size=256`、`VQ_SPLIT`	fixture 默认值；测试建队时再次明确传参	golden 实际只建立 pair 0 的 TX 队列
driver feature mask 全 1	`default_driver_features`	表示驱动愿意取设备声明位，不是所有特性都已启用
device features `0x0000000100000001`	fixture 的 `device_features`，注入 MODEL device	只有 CSUM 与 VERSION_1
GPA 随机分配、IOVA 随机分配	`HOST_MEM_RANDOM`、`IOMMU_IOVA_RANDOM`	地址例子必须用符号或运行日志，不能假定顺序连续

因此本次没有协商 PACKED、MQ、CTRL_VQ、MRG_RXBUF、EVENT_IDX 或 NOTIFICATION_DATA。`default_rx_mode=RX_MODE_MERGEABLE` 是行为配置字段，不能覆盖设备 feature 寄存器。不要据此给 golden 画出 mergeable RX 或 packed ring。

6.2 谁决定 BDF、BAR 和 qpair

`dpu_common` 负责设备/资源快照，Virtio 消费冻结结果。fixture 的 `end_of_elaboration_phase` 取得 `device_snapshot` 和 `resource_snapshot`，要求两者 frozen，查询 `get_expected_af` 得到 Function key，再用 `get_pcie_id` 得到 PCIe identity。它按 BDF 查找 `function_instances`，不是把数组第 0 项当成身份权威；随后 `list_bars` 必须得到三对 BAR。

`snapshot_is_frozen()` 还验证 resource snapshot 确实引用这一份 device snapshot。这里的“同一对”是对象关系，不是“字段看起来相似”。`build_flow` 检查的是已经接好的环境和 readiness，不能拿它当成分配 BDF、写 BAR 或完成设备初始化的函数。

`virtio_resource_client::bind_to_service(device_snapshot, resource_snapshot, service_key, why)` 是直接绑定入口；生产路径通过已经验证的 `vio_snapshot_ctx` 调用 `bind_to_ctx`。两者最后进入 `bind_validated`：service 必须为 `DPU_SERVICE_VIO_NET`，其 owner 必须与 service 的 Function key 一致，且资源快照至少含一条该 service 的 qpair binding。它逐条复制 placement 结果并按 `local_pair` 排序，再一次性提交内部映射。

每条映射保存 `virtio_pair_index`、`local_pair`、RX/TX 的 local virtqueue ID，以及 global qpair ID。关键关系是 **RX 和 TX 共用一个 global qpair ID**。global ID 标识 DPU 内部那一对资源；local virtqueue ID 才区分 Function 内收发方向。禁止将 global ID 乘二生成 RX/TX 队列号，也不能把 sorted array index、local pair 和协议 pair index 混为一个编号。

fixture 的 `tx_queue_id_for_pair(0, tx_queue_id, why)` 导出排序后的映射，检查索引范围，再返回该项的 `tx_virtqueue_id`。默认标准布局是 RX0/TX1、RX2/TX3，但测试仍使用映射查询。这个做法使 placement 更换 local 编号后，测试不必偷偷承担重新分配资源的职责。

绑定有明确生命周期：同一个 snapshot 对和同一个 service 的重绑返回成功；改绑到另一对快照或另一 service 被拒绝。`reset_runtime_state` 只清运行就绪，不释放 placement 所有权。`mark_runtime_ready` 表示绑定后的运行准备状态，不等价于 `DRIVER_OK`，更不等价于所有 ring 都已建立。

源码：resource client，L35、映射导出与就绪，L156、TX 队列查询，L537。

6.3 资源配置与驱动行为配置不是同一张表

`virtio_net_env::connect_phase` 为每个 Function 调用 `resource_snapshot.list_vio_bindings_for_service`，取实际 binding 条数为 `allocated_qpairs`。零条直接 fatal。随后调用 `cfg.get_service_config(service_key, allocated_qpairs, driver_cfg, why)`，将结果交给该 Function 的 `drv_cfg`。

`get_service_config` 优先使用显式 service 配置；没有覆盖时调用 `make_default_driver_config(max_pairs)`。默认的 num pairs 被实际分配上限截断，显式配置超出分配数也被截断，`max_vio_net_qpairs_per_device` 写入该 service 的可用上限。queue size、ring type、feature mask、RX 行为、IRQ 模式、NAPI budget 等才来自 Virtio 环境默认值。因此“配置要求两对”是驱动愿望，“placement 分了几对”是可用资源，设备的 `num_queues` 又是后面通过寄存器读到的能力，三者需要分别核对。

本次 fixture 覆盖 `default_num_pairs=2`，设备模型 `max_pairs` 也取这个值，`queue_max=256`，但模型的 feature mask 是独立字段；不能因为模型暴露四条数据队列，就自动得到 MQ feature。只有控制队列命令成功，才能把标准设备 active pairs 从复位后的默认一对改成多对。

私有 service 配置存在时，`get_service_config` 会验证它，将队列格式设为 `VQ_CUSTOM`、中断设为 polling、driver features 清零，并采用私有 RX 容量；没有显式 service 行为覆盖时才采用私有默认 ring 深度。这是按 Function 选择的协议，不是把标准 split 的类名替换为 custom 就完成适配。

源码：环境按资源解析行为，L433、默认行为生成，L197、service 覆盖，L339。

7 Transport 接通以后，怎样发现设备并完成协商

7.1 接通对象不是完成寄存器初始化

Function 的 `apply_function_configuration` 把 snapshot 的完整 PCIe identity 交给 transport，设置 VF 属性，把同一 host/BDF 注入 queue manager 和数据面 engine，再导入 BAR leases 与 resource client。`commit_validated_pcie_components` 把共享 Host memory、IOMMU、barrier、错误注入和 wait policy 接到 manager；把 transport、manager、memory、IOMMU 接到 ops；再让 FSM 引用同一个 ops 和 driver config。PCIe RC sequencer 是借用的外部传输入口。

这个连接步骤没有分配 ring，也没有生成 DMA 完成。标准协议绑定 `virtio_common_cfg_backend`；私有协议绑定自己的 backend。上层 ops 因而可以保持“复位、协商、建队”等动作名称，而寄存器契约由 backend 决定。preflight/commit 的分离用于避免一个 Function 验证失败后，环境留下半套已经可运行的连接。

源码：连接提交，L373、身份应用，L588。

7.2 从 Fabric BAR 租约到 capability 中的实际窗口

golden 的 `start_driver_flow` 先等待 protocol event interface 退出平台复位，然后调用 `transport.discover_and_init_bars()`。入口委托 `virtio_fabric_preconfigured_discovery::discover`，而不是重新枚举和分配全局 BAR 空间。

发现策略首先要求 Function 已受 Fabric 管理、BAR role layout 已激活、resource client 非空。随后将 requester identity 及 bar 引用接好，经配置事务 `program_fabric_bar_pairs()` 写三对 64 位 BAR；再读改写 PCI Command，把 Memory Space Enable 与 Bus Master Enable 置 1。BAR 已经有地址，不等于设备已经允许 MMIO、DMA 和 MSI-X：Command 位就是这个差别。

`cap_mgr.discover_capabilities()` 从配置空间 0x34 的 capability pointer 开始，按链表读配置 DWORD。它屏蔽指针低两位，识别 vendor-specific Virtio capability 与 MSI-X capability，最多走 48 项；达到上限报告可能存在环。Virtio capability 提供 `cfg_type`、BAR 编号、offset、length，notify capability 还带 multiplier。这里通过 PCIe 配置读得到的内容由设备模型回答，不是 fixture 把 cap manager 的 found 标志直接设为成功。

随后 `capabilities_use_fabric_windows` 核对角色：common/ISR/device/pci-config capability 应落在 device-memory BAR；notify 应落在 mailbox BAR；若有 MSI-X，其 table BIR 应落在 MSI-X BAR。common、notify、ISR 是必须发现的。角色错误或缺失会 fatal，不能继续用默认 offset 访问寄存器。成功后设置 `fabric_capability_discovered` 并让 resource client `mark_runtime_ready`。

需要区分两个层次：快照提供“这对 BAR 被授权放在哪儿、承担什么角色”；capability 说明“设备在该 BAR 内的哪个 offset 提供哪一种接口”。两者结合才能形成 MMIO 地址。notify 地址同样不能简单写成 `BAR + queue_id*4`，其公式是 `BAR_base + notify_cap.offset + queue_notify_off* notify_off_multiplier`。

源码：发现策略，L97、capability walk，L116、MSE/BME，L760。

7.3 status、feature mask 和失败返回

发现之后，fixture 调用 `ops.device_reset_verified(step_ok)`。它通过 backend 确认设备 status 回到 0，确认后才回收上一生命周期的队列/DMA；超时不能当作已停止 DMA。失败时 fixture 返回 `ok=0`，不会继续写 ACKNOWLEDGE。

随后依次执行 `set_acknowledge()`、`set_driver()`、`negotiate_features(driver_cfg.driver_features, negotiated)`、`set_features_ok(step_ok)`。标准 backend 最终进入 transport：先写 DFSELECT=0/1 分别读两个 32 位 device feature 窗口，得到 64 位 `dev_feat`；计算 `negotiated = dev_feat & driver_supported`；再通过 GFSELECT=0/1 写低/高半部。transport 缓存 device/driver features，并更新 `notification_data_enable`；ops 也保存 `negotiated_features`，后面的 header 长度和控制队列判断使用它。

`FEATURES_OK` 必须写后读回，设备清掉该位就是拒绝。fixture 此时调用 `set_failed` 并退出。本次全 1 的驱动 mask 与模型的 CSUM|VERSION_1 相交，仍然只有这两位。队列类型是显式配置参数，`virtqueue_manager` 并不会从 feature 位自动挑选 split 或 packed。因此设计新的 packed 测试时，需要同时保证设备广告、驱动协商结果和队列类型一致，不能只改一个 enum。

fixture 接着读取 `num_queues`，计算 `2*num_queue_pairs`，若协商 CTRL_VQ 再加一；所需数超过设备能力则写 FAILED 并退出。本次检查四条队列容量，但并未在这里创建四个 ring。它先 `setup_msix(4)`，再写 DRIVER_OK 并回读确认；最后把共享 FSM 的只读生命周期视图置为 `FSM_READY`，设置 `m_driver_flow_started=1`。

正常 status 累加可读作 `0 → ACKNOWLEDGE → ACKNOWLEDGE|DRIVER → 加 FEATURES_OK → 加 DRIVER_OK`。不要用 FSM_READY 替代设备 status 的证据：fixture 已显式回读 DRIVER_OK，但其设置 FSM_READY 不会自动创建队列、启动 background worker，亦不会替设备写 used ring。

源码：verified reset 与 status，L14、特性交集，L321。

7.4 与 full_init 的对照：不是同一次调用

`virtio_auto_fsm::full_init()` 是批量生命周期入口。它先拒绝 unknown、零或超出 Function 上限的 queue-pair 数；`do_full_init` 推进 DISCOVERING、NEGOTIATING、MSIX_SETUP、QUEUE_SETUP。与 fixture 最大差别是，它调用 `setup_all_queues(active_num_pairs, queue_format(), queue_size, setup_ok)`，成功后才写 DRIVER_OK 并进入 READY；fixture 则先完成 status/MSI-X，再由测试显式建一条 TX 队列。

复位失败处理也不能混为同一保证：do_full_init 的 reset 分支，L281 仅对 `protocol.requires_verified_reset()` 为真的协议立即检查 reset_ok，失败进入 FSM_ERROR 并返回；标准协议调用 `ops.device_reset()`，该包装不把确认结果传回 FSM，随后仍进入 ACKNOWLEDGE/DRIVER 步骤。fixture 则始终显式调用 verified 接口并检查结果。因此 fixture 的“reset 失败立即退出”不能推广为本基线标准 full_init 的同等保证。

`setup_all_queues` 通过 `setup_queue_range` 建 RX/TX 成对队列，按协商结果额外建立 control VQ。CTRL_VQ 存在而 MQ 未协商时，多于一对会与标准 control queue 布局冲突，因此在建队前拒绝。批量失败会尝试回滚本次已建队列；若标准设备未协商 RING_RESET，或单队停队不能确认，会升级为 verified Function reset，影响整个 Function。Function reset 也失败时，保留未能安全停止的资源并置 recovery pending，不能保证失败后一定全部释放。

MQ 的 `active_num_pairs` 还表示 Host 预建的目标拓扑，不能孤立解释为设备已接受值。多对标准队列建好后 `mq_pairs_committed` 仍为假，必须经过 `configure_mq` 的 control 命令确认。`start_dataplane` 会拒绝未提交的目标。这就是“预留两对资源”“预建两对 ring”“设备 active_pairs=2”三个状态的实际代码分界。golden 默认没有 MQ，本章不把该测试说成已经证明多队列启用。

源码：full_init，L228、start_dataplane 门槛，L357。

8 建立 ring：对象、Host GPA 与设备 IOVA

8.1 setup_queue 才是 ring 的分配入口

测试的 `flow.setup_queue(tx_queue_id, 256, VQ_SPLIT, ok)` 先要求 flow 已启动且 Function 上下文完整，再调用生产 `ops.setup_queue`。fixture 没有自己分配 descriptor，更没有预先放一个完成条目。ops 初始 `ok=0`，先拒绝 recovery pending、缺失 transport/manager/IOMMU/memory，以及重复 queue ID；重复检查发生在查询设备与分配内存之前，避免覆盖旧 ring 所有权。

标准 backend 选择 queue ID 并读取 Q_SIZE 上限。请求 0 表示采用设备上限；超过上限则警告并裁剪；有效大小为 0 则失败。本次请求和模型上限都是 256。接着 manager 按 enum 创建 `split_virtqueue`、`packed_virtqueue` 或 `custom_virtqueue`，调用其 `setup` 注入共享对象和 host/BDF，登记到 `queues[queue_id]`。manager 此时只创建对象，并未调用 `alloc_rings`。

ops 验证队列 host 与 transport 的冻结 DMA host 一致，生成 `virtio_queue_program_t`，填 host、BDF、queue ID、有效大小、格式、local ID 与方向，再让 backend `prepare_queue` 补充协议信息。私有 backend 在这里配置方向、local qid、RX 容量与 doorbell。通过后才调用 `vq.alloc_rings()`。

manager 的 `create_queue` 单独遇到重复 ID 会报错并返回旧对象；正式 ops 则提前拒绝，不能把 manager 的行为当成“setup_queue 幂等成功”。创建失败返回 null，域不一致或 prepare 拒绝时销毁新对象，调用方仍得到失败。

源码：生产建队入口，L15、manager 工厂，L63。

8.2 split 256 的三块布局

`split_virtqueue::alloc_rings()` 向共享 Host memory manager 做三次分配。下表是源码要求的逻辑字节数与对齐；底层 buddy allocator 的实际保留粒度可以更大，不能把逻辑长度当成物理分配器占用统计。

区域	大小公式	N=256	对齐	后续写者
descriptor table	16N	4096 B	4096 B	驱动发布 addr/len/flags/next
available ring	6+2N	518 B	2 B	驱动发布 head 与 avail index
used ring	6+8N	2054 B	4096 B	设备发布完成 id/len 与 used index

descriptor 项按小端存储 64 位地址、32 位长度、16 位 flags、16 位 next。avail 包含 flags、idx、N 个 16 位 head 和末端 event 字段；used 包含 flags、idx、N 个 `{id,len}` 以及末端 event 字段。源码无论本次是否协商 EVENT_IDX 都预留这些尾部字节，所以公式中的常数是 6。

三块成功后清零，然后为 descriptor 的空闲链填写 next；`free_head=0`、`num_free=256`、`avail_idx=0`、`last_used_idx=0`，状态进入 `VQ_CONFIGURE`。此时初始化 used 区为零是 Host 在交付之前准备内存，不是测试伪造设备完成；后续非零 used 完成仍必须来自设备 DMA。

由于 Host 分配策略随机，不画“desc 后面紧接 avail 后面紧接 used”的假连续布局。应把它们命名为 `G_desc`、`G_avail`、`G_used`，分别记录 GPA、大小和对齐。请求失败用全 1 地址标识。这里有一处阅读限制：split/packed 的 `alloc_rings` 只报告错误，没有 success 返回值；其 `get_ring_regions` 仍按成员地址返回固定数量的区域。因此不能仅凭 ops 的“空区域列表检查”就声称标准 ring 的分配失败已被完整拦截；golden 的成功路径并未覆盖内存耗尽故障。

源码：split 分配，L117。

8.3 GPA 与 IOVA 分开追踪

GPA 是共享 Host memory backing 的地址，Host 侧 `mem_write/mem_read` 使用它。IOVA 是设备 DMA 和 descriptor 中使用的地址，由 IOMMU 在 host/BDF 域内映射；BAR 地址则是寄存器访问窗口。这三种地址即使数值偶然相同，也不是同一种资源。

fixture 的 GPA aperture 为 `0x0000000100000000..0x00000001ffffffff`；默认 IOVA aperture 独立配置为 `[0x0000000080000000, 0xfffffffffffff000)`。`iommu_strict=1` 启用权限检查。binding 给 PCIe Host memory 路径的 manager 与驱动分配 ring 的 manager 相同，因此设备经 IOMMU 翻译后的 DMA 落到同一份 backing，不存在一个“驱动内存副本”和另一个“设备内存副本”。

ops 调用 `get_ring_regions(region_gpas, region_sizes)`，逐区 `map_for_host(transport.iommu_host_id(), transport.bdf, GPA, size, DMA_BIDIRECTIONAL)`。ring 使用双向权限，是这一实现对整个 ring 区的统一映射策略；普通 TX payload 与 RX buffer 的权限在数据面另外指定。mapping 返回 0 或全 1 算失败，已成功映射的部分会 unmap，随后销毁队列并释放 ring。

成功后 `ring_iovas[queue_id]` 记录映射所有权，`ring_iova_of` 按“所在 region 的 IOVA + 区内偏移”计算 desc/driver/device 三个设备地址。此时写寄存器用的是 `I_desc/I_avail/I_used`，不是 `G_desc/G_avail/G_used`。ring 对象负责 Host 分配的释放，ops 负责对应 IOVA 的撤销；reset/teardown 必须先确认设备不再 DMA，再共同回收。

源码：地址域默认值，L71、IOMMU 映射，L178。

8.4 packed 与 DPU 私有 ring 的边界

packed 仍是标准 Virtio 格式。`packed_virtqueue::alloc_rings` 一次申请 `16N+8` 字节、4096 对齐的连续 Host 块：前面是描述符，后面是各 4 字节 driver event 和 device event。N=256 时逻辑大小 4104 B。它把 `driver_ring_addr/device_ring_addr` 指向两个 event 结构，初始化 avail/used wrap 都为 1，index 为 0。它没有 split 的两个独立 head/index ring；描述符通过 AVAIL/USED 位和 wrap 表达归属。

虽然 packed 只做一次 Host 分配，其 `get_ring_regions` 明确返回三个映射区域：`16N`、4、4。不要据物理连续性擅自推导 event IOVA 等于 descriptor IOVA 加长度；当前 ops 按三个区域各做一次映射并编程三个地址。释放 Host 块一次，与撤销三个映射，是两个不同粒度的动作。

`VQ_CUSTOM` 对应 DPU 私有协议，不属于标准 packed 的别名。custom ring 同时包含私有 descriptor 与 CQ：TX 用 `16N+2` 的有效字节，RX 用 `32N`，再向上取整到 4096 分配。ops 对显式私有深度要求 2..4096 且为 2 的幂；custom 分配还要求 local qid 不超过 255，禁止重复分配，并冻结方向、local ID 与大小。它预留一个槽，空闲上限为 N-1；标准 split 的 256 个空闲 descriptor 不能直接照搬成 custom 可提交 256 项。

私有协议通过独立 backend 编程 VTX/VRX 等寄存器和私有 doorbell，无标准 feature 协商、无 Virtio-net header，完成方式为 polling。golden fixture 的标准 status/capability 路径不能未经调整就作为私有协议教程；私有实例应从 `examples/dpu_priv_ring` 的独立测试阅读。

源码：packed 分配，L155、packed 映射区域，L575、custom 分配，L206。

9 把队列发布给设备：寄存器、通知、中断与 RX 预填

9.1 队列 enable 是发布边界

映射完成后，ops 从 notify manager 获取该 queue ID 的 MSI-X vector；初始化已规划的向量直接复用，后续新队列可动态分配。返回 `0xffff` 表示无可用向量，此时撤销本次映射、删除记录并销毁队列。随后将三个 IOVA、ring 总字节和 vector 填入 `qprog`，调用 backend 的 `configure_queue` 和 `enable_queue`。

标准 `virtio_common_cfg_backend::configure_queue` 进入 `transport.setup_single_queue`，顺序是选择 queue、写 size、写 descriptor/driver/device 64 位地址、写并回读队列 MSI-X 映射、读取 queue_notify_off、最后写 queue_enable=1。这个顺序确保设备拿到 enable 时，地址和向量都已经准备。设备拒绝 vector 时，notify manager 记录实际读回的 NO_VECTOR，不能只保留“驱动想写的向量”。

标准 backend 的 enable 操作本身不再发 TLP，因为 configure 的末尾已经 enable。它也没有通用的配置失败返回通道：`configure_queue` 调完 transport 后置 `ok=1`；底层寄存器错误由 transport 的报告体现。fixture 因此额外选择该队列并读 queue_enable，发现未使能就把自身 `ok` 清零。文档不能把这一层的 success 描述成“逐个地址和所有硬件字段都已完整回读验证”。

ops 成功后保存 `queue_programs[queue_id]`，设置 `vq.state=VQ_ENABLED` 与 `queue_enable=1`。这些是 Host 侧记录；设备模型是通过真正收到的 common-config 写建立设备侧队列状态。后面设备读 descriptor 时，依赖的是寄存器中的 IOVA 与 IOMMU 翻译，并不直接拿 manager 内的 SV 队列句柄读取完成状态。

源码：标准 backend，L184、寄存器发布，L694、fixture 回读，L830。

9.2 MSI-X 表、queue vector 与 notify 是三件事

`ops.setup_msix(num_queues)` 仅在 manager 已有队列数恰好等于 num_queues 时使用实际队列 ID；数量不等时（包括尚未建队或只建了一部分），根据目标数量生成 ID 集合，存在 control VQ 时单独计算其位置。backend 检查 MSI-X capability，初始化表、分配中断向量、绑定 config vector 与每队列 vector、解除 mask，并在 PCI 配置空间置 MSI-X Enable、清 Function Mask。资源不足时通知管理器可以退为共享 MSI-X 或 INTx；无 MSI-X capability 时也退为 INTx。因此 `IRQ_MSIX_PER_QUEUE` 是请求/默认策略，运行时实际模式仍需看 manager 的结果。

队列 kick 是 Host→设备的通知写，MSI-X 是设备→Host 的完成/事件通知，不能因为名字里都有 notification 就把二者当一条总线动作。golden 默认未协商 NOTIFICATION_DATA，`transport.kick` 写 2 字节 queue ID；协商后才写 4 字节，其中 split 包含 next avail index，packed 还带 wrap。地址使用第 7 章的 capability 公式，`queue_notify_off` 来自设备读回。未建队导致 offset 数组越界时，kick 报错并返回。

queue enable 自身不是提交包；它只是让设备认识 ring。TX 的后续 `add_buf` 发布 descriptor/avail，再按 suppression 决定 kick；设备自行读取 ring、搬运数据、发布 used、发中断。测试不能用一次 notify 写的发生代替 DMA/used/MSI-X 整条路径成功的证据。

源码：MSI-X 规划，L326、标准 MSI-X 配置，L211、kick，L647。

9.3 DRIVER_OK 之后谁预填 RX

golden TX 测试到此只有一条 TX ring，不会自动补 RX。独立 RX 测试会建立 RX 队列，先把四帧注入 MODEL 网络侧，再显式 `refill_rx(queue_id, 4, ok)`；这是另一次测试运行，不是 golden TX 的下一步。

完整 FSM 路径则在 `full_init` 成功后进入 READY；只有显式 `start_dataplane()` 才检查 MQ commit 与前一代 worker 退出情况，转 RUNNING，为每个偶数 RX 队列调用 `ops.rx_refill(rx_qid, drv_cfg.queue_size)`，然后启动背景补充、完成和中断等 worker。因此 RX 预填不属于 `full_init` 的 ring 建立阶段。

`rx_refill(queue_id, num_bufs, buf_size=0)` 从 manager 取队列，用 `vq.net_hdr_bytes(negotiated_features)` 决定 header 长度；容量优先采用显式非零 buf_size，其次采用 backend 给队列的 RX 容量，两者都为 0 才退到 header+1514。DMA 分配大小向 4 字节取整，descriptor 长度仍是可用容量。每个 buffer 分配 Host GPA、清零，以 `DMA_FROM_DEVICE` 映射 IOVA，创建一段 device-writable SG，经 `vq.add_buf(sgs, 0, 1, null, 0)` 发布。成功后所有权转入 `rx_dma_map`，供 RX 完成或 reset 回收；有提交且需要通知时才 kick。

队列满、内存不足、映射失败或 add_buf 失败都会使循环提前结束。此 task 没有“实际填了多少”的 output 参数；fixture 的 `refill_rx` 只要找到 ops 并完成调用就把 `ok=1`，所以不能把这个 ok 当成精确填满请求数的断言。要证明四帧被接收，还要检查设备完成与生产 `rx_receive` 的返回数量。

在这一步，驱动提供的是空 buffer 与写权限；网络帧与 used 完成仍由设备模型生成。清零 RX buffer、填 descriptor、更新 avail 都是驱动行为，设备通过 EP-originated DMA 写 header/payload/used 才是接收完成。第 12 章从这一已发布的 ring 接续逐字节 RX 数据路径。

源码：RX 预填，L238、fixture refill，L984。

9.4 关键函数导航

下表中的路径均相对 `virtio_net_vip/`；上文链接固定到本文基线 SHA，避免分支继续演进后行号漂移。

源码入口	主要调用者与参数	结果和应检查的边界
`tests/virtio_real_driver_flow_fixture.sv:307 end_of_elaboration_phase`	UVM；无业务输入	从 frozen snapshot 解析 AF/BDF/BAR，缺失句柄或快照不冻结不能 ready
`src/sriov/virtio_resource_client.sv:70 bind_validated`	bind_to_ctx / bind_to_service；snapshot 对、service key	按 local_pair 保存映射；拒绝错误 owner、空 binding、改绑
`src/env/virtio_net_env_config.sv:339 get_service_config`	env.connect_phase；service key、allocated_qpairs	输出 driver_cfg，按资源限制对数，叠加私有协议选择
`src/sriov/virtio_function_instance.sv:373 commit_validated_pcie_components`	Function PCIe commit；共享资源与 ops/FSM 候选	绑定 backend、域与外部 sequencer；不分配 ring
`src/transport/virtio_discovery_strategy.sv:97 discover`	transport.discover_and_init_bars；transport	编程 BAR、开 MSE/BME、扫描并检查 capability，错误 fatal
`src/transport/virtio_pci_cap_manager.sv:116 discover_capabilities`	发现策略；bar_ref	配置链表扫描，48 项上限，记录各窗口
`tests/virtio_real_driver_flow_fixture.sv:736 start_driver_flow`	golden run_phase；ref ok	verified reset、协商、容量、MSI-X、DRIVER_OK；不建 ring
`src/agent/virtio_auto_fsm.sv:228 full_init`	另一条 FSM 初始化入口；drv_cfg	批量建队后 DRIVER_OK，标准多对仍需 MQ commit
`src/agent/atomic_ops/virtio_atomic_ops_queue.svh:15 setup_queue`	fixture/FSM；qid、size、type、output ok	创建→分配→映射→编程；重复/恢复 pending 提前拒绝
`src/virtqueue/virtqueue_manager.sv:63 create_queue`	ops；qid、size、type	类型工厂与共享引用注入，登记队列对象
`src/virtqueue/split_virtqueue.sv:117 alloc_rings`	ops；队列已有 size/mem	三块 GPA、清零与 free list，分配失败报告
`src/virtqueue/packed_virtqueue.sv:155 alloc_rings`	ops；队列已有 size/mem	一块 GPA、descriptor+两个 event，wrap 初值为 1
`src/iommu/virtio_iommu_model.sv:178 map_for_host`	ops；host/BDF/GPA/size/权限	返回设备 IOVA，记录域与权限映射
`src/transport/virtio_pci_transport.sv:694 setup_single_queue`	common backend；三地址、大小、vector	地址和 vector 先于 enable，保存设备 notify offset
`src/transport/virtio_pci_transport.sv:647 kick`	backend；qid、next index、wrap	按协商选择 2/4B 通知，拒绝无 notify offset 的 qid

源码入口	主要调用者与参数	结果和应检查的边界
`src/agent/virtio_auto_fsm.sv:357 start_dataplane`	显式 FSM 数据面启动	READY 与 MQ commit 门槛，预填 RX 后启动 worker
`src/agent/atomic_ops/virtio_atomic_ops_datapath.svh:238 rx_refill`	fixture 或 FSM；qid、数量、可选容量	发布可写 buffer，部分成功可提前结束，无数量 output

10. 具体业务：一个 16 字节 TX 如何提交

10.1 先看测试真正调用了什么

入口在 `virtio_net_vip/tests/virtio_real_driver_flow_test.sv`。其 `run_phase():40` 先验证 fixture、取得 pair0 的 TX qid，调用 `start_driver_flow()`，再 `setup_queue(tx_queue_id,256,VQ_SPLIT,ok)`。MODEL 分支从第 84 行开始，实际请求可节选为：

```
pkt = virtio_real_driver_flow_packet::type_id::create("tx_packet");
hdr = '{default: 0};
flow.submit_tx(tx_queue_id, hdr, pkt, 1'b0, desc_id, ok);
flow.complete_tx(tx_queue_id, completed, 1, completed_count, ok);
```

此处 `use_indirect=0`，不是 indirect descriptor 测试；completion budget=1，不是等待所有队列清空。`pkt` 的 new():16 与 do_pack():24 在同一文件，构造函数保存以下确定的 16 字节，do_pack 按每字节 8 bit 打包：

```
de ad be ef 01 23 45 67 89 ab cd ef 10 20 30 40
```

这是用于 golden lifecycle 的合成 UVM 对象，不是一份完整合法的以太网报文。生产 TX 操作仍会把它的 16 字节作为 payload 投递；不能因为方法名里有 packet 就声称该测试证明了 Ethernet/IP/UDP 解析正确。只有空打包结果才触发 64 字节最低容量兜底，这个对象有数据，不会变成 64 字节。

`flow` 只是把测试意图交给已绑定 Function 的生产 ops。它不在 fixture 中直接写 desc/avail/used，更不自行调用设备 push_used 来制造通过。

10.2 submit_tx 的对象与失败契约

fixture.submit_tx():880 的输入为 `queue_id`、`net_hdr`、`pkt`、`use_indirect`，通过 ref 参数返回 `desc_id`、`ok`。开始先写 `ok=0`、`desc_id='1`；只有已有 `m_flow_ready`、已开始 driver flow、非空 pkt、可解析的 selected Function 和可用 ops，才调用该 Function 的 `ops.tx_submit()`。

返回后 `ok=(desc_id!='1)`。它表明驱动接受并发布了这一条提交，不表明设备已经完成 DMA，更不表明中断已经被 monitor 采样。直接调用 ops 的调用者也须保证这些引用已绑定；生产 ops 并不会为一个裸对象重建完整环境。

实际函数 virtio_atomic_ops.tx_submit():22 位于 `agent/atomic_ops/virtio_atomic_ops_datapath.svh`。这个文件被 `agent/virtio_atomic_ops.sv` 在 class 体内 include：它不是另一个 UVM 组件，也不是另一个数据面对象。去找函数时应先确认所属类，再打开被 include 的实际实现文件。

tx_submit 先把 desc_id 保持为失败哨兵。请求 indirect 却未协商 INDIRECT_DESC、qid 没有对应 vq、无 header 的队列却携带 offload 请求，都会提前报错返回；这些情况不会进入成功 DMA 所有权记录。

10.3 header 和 payload 的字节从哪里来

主线 negotiated features 是 CSUM|VERSION_1。`vq.net_hdr_bytes()` 调用 virtio_net_hdr_util.get_hdr_size():27：HASH_REPORT 为 20 字节，MRG_RXBUF 或 VERSION_1 为 12 字节，其余为 10 字节。因此这里是 12，而不是“未协商 MRG_RXBUF，所以必然是 10”。

`pack_hdr():54` 按小端把 flags、gso_type、hdr_len、gso_size、csum_start、csum_offset，以及本格式携带的 num_buffers 编码。输入 hdr 全零，得到 12 个零字节；CSUM 只是允许能力，零 flags 没有请求实际 checksum offload。

payload 先试 virtio_net_packet_adapter.pack():22。如果对象是外部 `packet_item`，adapter 取真实 wire bytes，避免把它 UVM do_pack 的长度前缀当成网络帧。当前合成对象不是 packet_item，于是走 `uvm_packer → pkt.do_pack → get_bytes`，恰好取回上述 16 字节。

主线的 descriptor 长度与 backing 请求如下。分配器内部还可能按自己的 granule 向上取整，表中是 ops 请求的字节数，不把它等同于 Buddy 内部实际占用量。

区域	有效 descriptor 长度	ops 请求 backing	实际内容
Header	12 B	DWORD 对齐后 12 B	全零 virtio-net header
Payload	16 B	DWORD 对齐后 16 B	固定 16 B 合成数据

DWORD padding 的目的，是让设备模型经 PCIe MemRd 读最后一个不完整 DWORD 时不跨出实际 backing block。descriptor 长度仍是有效字节长度，不能把 padding 字节算进业务 payload。

10.4 每一次提交的 DMA 所有权怎么建立

tx_submit 创建一个局部 `virtio_dma_buffer_set("tx_dma")`，调用 `bind_domain(mem,iommu,transport.iommu_host_id(),transport.bdf)`。这个集合只借用 mem/IOMMU；它拥有本次提交申请的 GPA 和 IOVA 记录，不拥有共享 manager。

virtio_dma_buffer_set 中关键函数的使用顺序是：

函数位置	当前调用	输出或副作用
`new:24 / bind_domain:29`	空集合绑定 Host/BDF 域	保存借用引用，不分配 payload
`alloc:38`	header 与 payload 各一次	返回集合索引，登记 GPA；失败 -1

函数位置	当前调用	输出或副作用
`write:52`	写 12 B header、16 B payload	修改共享 Host backing
`map:63`	两项均 DMA_TO_DEVICE	建立设备可读 IOVA；0 或全 1 视为失败
`get_iova:78`	组装两个 SG entry	descriptor 用 IOVA，不用 GPA
`release_all:96`	提交前失败回滚	先解映射已映射项，再释放所有 GPA
`disown:107`	记录已交给 ops 后调用	只清集合，不释放 backing

先分配、再写内容、再建立 DMA_TO_DEVICE 映射。第二份分配失败会释放第一份；任一映射失败会解映射成功项并释放两份 backing；add_buf 失败也由集合回滚。不会把没有提交成功的记录加入 tx_dma_map，避免后来等待一个不存在的完成。

成功发布后 adopt_dma_set():272 把每项转成 `{host_id,bdf,gpa,iova}` 的 normal_dma_record，按 header→payload 顺序追加到 `tx_dma_map[qid]`，再 disown 集合。此后由完成或已确认的设备复位退休，不再由局部集合释放。

注意 backing 的所有权分两类：ring/indirect table 由 queue 持有；普通 header/payload 由 ops 的 DMA 记录持有。poll_used 归还 descriptor 并不意味着它可以释放这些普通 payload。

10.5 两个 SG 如何变成 Split descriptor 与 avail

tx_submit 构造两个 device-readable SG list：第一个 `{addr=I_hdr,len=12}`，第二个 `{addr=I_pkt,len=16}`。调用 `vq.add_buf(sgs,2,0,pkt,0)`：n_out=2、n_in=0，token 是原 pkt 句柄，indirect=false。

split_virtqueue.add_buf():203 先检查 SG 数相加溢出、SG 总数为 0 或超过输入数组长度、没有 entry、空闲 descriptor 不足；输入数组可以包含不参与本次提交的尾项，并不要求长度完全相等。直接链需要两个 ring descriptor，indirect 模式才只占一个主 ring 槽并另建表。

在这个新建、无其它提交的 queue 中，空闲链从 0 开始，所以 head=0，第二项=1；这是“新队列首提交”的结果，不是以后每次都返回 0。两个 descriptor 的业务含义如下：

Descriptor	地址 / len	Flags / next	Device 权限
0，链首	`I_hdr / 12`	NEXT / 1	可读，不置 WRITE
1，链尾	`I_pkt / 16`	0 / 0	可读，不置 WRITE

write_desc 在 Host GPA 的 desc table 编码每个 16 B 项：64 bit addr、32 bit len、16 bit flags、16 bit next。内存位置是 GPA，descriptor 里的 addr 字段则是 IOVA，必须分清这两个地址层次。

发布次序为：写完整 descriptor → `barrier.wmb()` → `avail.ring[0]=head` → `avail_idx` 从 0 加到 1 并写进 avail.idx → `barrier.mb()` → `token_map[head]=pkt` 及发布账本。具体字节偏移：avail ring entry 在 `G_avail+4`，avail.idx 在 `G_avail+2`。

两个 descriptor 只产生一个 avail entry，因为 avail 公布的是“一条链的 head”，不是每个 SG 分别公布。空闲 descriptor 数从 256 减到 254。buffer 数、descriptor 数、avail entry 数、packet 数不是同一个计数。

10.6 真正的通知由谁发，submit 返回又意味着什么

add_buf 不会自己发 PCIe notify。tx_submit 先 adopt DMA 所有权，再检查 `vq.needs_notification()`。主线未协商 EVENT_IDX，设备 used.flags 没有要求 NO_NOTIFY，首条提交需要通知。

调用链是 kick_with_fault():242 → PRE_NOTIFY 故障边界 → `vq.prepare_notify()` → `cfg_backend().notify_queue(qprog,next_idx,wrap)` → virtio_pci_transport.kick():647 → BAR write，然后 POST_NOTIFY 边界。只有 prepare_notify 返回真才调用后端通知，返回假也仍执行 POST_NOTIFY。普通 backend 的 `notify_queue():123` 转交 transport；私有 backend 使用自己的 doorbell，不能用标准 notify offset 替代。

没有 NOTIFICATION_DATA 时，写入的是标准 queue_id 通知；协商该 feature 才携带 next_idx/wrap 扩展数据。实际 MMIO 地址由 capability base、queue_notify_off 和 notify multiplier 组合，不能把“qid=1”直接当成绝对 BAR 地址。

`virtqueue_base.kick()` 只是队列层统计接口，并不保证发出 TLP。当前生产路径统一走 ops/配置后端的 notify；直接在测试里调用裸 vq.kick 而没有 transport 通路，不能证明真实设备已被唤醒。

到这里 driver 可以返回 desc_id，fixture 可以返回 ok=1；设备 worker 的处理是另一条并发执行路径。完成的唯一数据依据是设备在共享 Host memory 发布的 used 状态，下一章继续跟踪它如何出现。

11. 独立设备怎样消费 TX 并产生完成

11.1 notify 从 PCIe 到设备 worker

MODEL 的 RC→EP 请求由外部 PCIe 环境送到被 factory 替换的 `vdev_ep_driver`。它的 handle_request():35 先请共享 device_port.serve 判断请求是否属于某个注册的 vdev；处理不了才交回基类。`serve():89` 按 BDF 处理配置请求，按设备认领的 BAR 地址处理 MMIO，并按 PCIe Byte Enable 提取有效字节。

本次 notify 是 MMIO write：port 调用 `dev.mmio_write()`，继而 `vdev_virtio_pci.region_write()` 进入 notify 区域。没有 PCIe Memory Write Completion 等待；posted write 与设备 worker 处理不能等同为一个同步函数调用。

vdev_virtio_pci.notify():434 从 data 低 16 bit 取 qid，发布 notify 事件、增加通知计数，然后检查 qid 是否匹配 notify slot、是否越界、队列是否 enable、是否已有 DRIVER_OK。条件不足时告警并忽略调度。MQ 协商时，未激活 pair 的 TX 也不能因为 Host 建好了 ring 就发包。

检查通过后 `schedule(qid)` 把队列放入工作列表并触发 work_ev；同一个 q 已在列表时不重复入队。设备基类 run_phase():380 是常驻 worker：优先处理待投中断，再 pop 队列并调用 `service_queue(q)`。这是设备侧 run task，不是顶层 virtio env 自动初始化的任务。

11.2 设备看到的 ring 地址怎样访问同一份 Host 内存

service_queue():491 保存当前 epoch，循环要求 epoch 不变、queue enable、未 broken、DRIVER_OK。RX 还要求有待收帧。每轮执行 `fetch → chain_ok → consume → push_used`，任何失败、复位换代或中途停队都不会继续正常发布这一轮完成。

设备的 desc/driver/device 地址来自驱动已编程的 common config，是 IOVA。`dma_rd/dma_wr` 先检查 PCI Bus Master Enable，再交给 port；它们不从 queue 对象中偷取 GPA，也不使用驱动 token_map 来猜 payload。

device_port.dma_read():192 与 dma_write():219 按 IOVA 和翻译后 GPA 的双侧 4 KiB 边界切块。`page_chunk():267` 用 `{m_host_id,requester BDF}` 和 DMA 方向调用共享 IOMMU；映射缺失或权限不允许时报告 VDEV_DMA，返回失败，不绕过 IOMMU。

MODEL 的翻译发生在 port 内：IOVA→GPA 后，懒创建的 virtio_pcie_model_dma_adapter.read():120 / write():181 把 GPA 与 requester BDF 转成 EP-originated PCIe MemRd/MemWr。adapter 借用 EP driver，DWORD 对齐、Byte Enable、Completion 取回由这一层处理；单次请求不得跨 4 KiB，读操作还受超时限制。

PCIe RC 服务访问的是 fixture 预先绑定的同一个 host_mem_pool manager，因此设备读到驱动写过的 descriptor/header/payload；设备写 used 后，驱动也读到同一份 backing。这里没有第二套 payload 镜像替设备制造结果。

REAL_DUT 不能照搬这一步的地址含义：RTL 发出的 TLP 本身可以携带 IOVA，本地 Host responder 使用 IOVA proxy 完成翻译。MODEL port 先翻译后发出的 TLP 则携带 GPA。不要以 MODEL monitor 看见的 GPA 为理由，把 REAL_DUT 的 IOVA 直接当 GPA 使用。

11.3 从 avail.head 到 payload 字节

fetch():523 在 split 分支先 DMA 读取 `I_avail+2` 的 16 bit avail.idx。本次读到 1，设备 last_avail 初始 0，表示有一条新链；若两者相同，函数返回 got=0，不能凭 notify 就伪造工作。

它检查生产者领先数不得超过 queue_size，再读 `I_avail+4+2*(last_avail%256)`，得到 head=0。随后推进设备 last_avail，并经 `read_split_chain():642` 从 `I_desc+16*head` 逐项读 descriptor，沿 NEXT 找到第二项。越界、循环、非法间接表或 DMA 失败会由 device_error 处理，而不是按正常完成归还。

`chain_ok():701` 检查零长度、可写之后再出现可读、TX 出现 WRITE、RX 链首非 WRITE 等方向错误。本次两项都可读，长度分别 12 和 16，合法进入 `consume():741` 的 TX 分支。

`read_readable():725` 分别 DMA 读 I_hdr 的 12 B 和 I_pkt 的 16 B，按链顺序拼成 28 B。`tx_frame():800` 解前 12 B 的 header，发布 descriptor-chain 事件，再把余下 16 B 发布为 VDEV_PKT 的 TX bytes。它并不重新对原 pkt 句柄 do_pack，而是使用实际 DMA 读回的字节。

模型还调用 `inject_rx(q/2,frame)` 将帧放进同对 RX backlog。当前 TX 测试没有建立 RX0 ring，所以这份回环帧不会被自动交给驱动，也不能算作已验证 RX。后续整设备 reset 会清 backlog；第 12 章的 RX 测试是另一轮独立激励。

11.4 一条链如何写成一个 used entry

TX 两个 descriptor 产生一个 used entry，不是每个 descriptor 产生一个完成。`consume()` 的 TX 分支不向任何可写业务 buffer 写入，所以本模型的 used_len 保持 0；它不是“发送 payload 长度=16”的计数器。

push_used():948 先把 `{id=head=0,len=0}` 小端写到 `I_used+4+8*(used_idx%256)`，再将设备 used_idx 从 0 加到 1，DMA 写 `I_used+2`。写回成功才增加 completion_count、发布 VDEV_COMPLETE，然后 `used_notify():1004` 决定是否中断。

没有 EVENT_IDX 时，设备读 avail.flags 的 NO_INTERRUPT；主线没有禁用 callback，因此按已绑定队列 MSI-X vector 触发中断。IRQ 被 mask 时可先挂起，不能保证每个 used 都立即产生一个可见中断。

`vdev_base.raise_irq():354` 按 MSI-X Enable、表项及 mask 判断能否发送，增加设备中断计数；port.raise_msix():241 构造 MSI-X Memory Write 供 PCIe monitor 观测。MODEL 这里延迟发布被观察的事件，不阻塞 worker。故“设备 interrupt_count 已增加”和“monitor 已采到中断”可能不在同一时刻。

11.5 驱动读取 used，然后分两层回收

fixture.complete_tx():918 清空 output 队列，有限次调用 `ops.tx_complete(qid,completed,max_budget)`；本轮没有新结果就等 1 ns，直到看到新增对象或轮询次数上限耗尽。max_budget 是单次 ops 的完成数量预算，`m_poll_timeout_ns` 才限制 fixture 的尝试次数。它不会等待中断后才允许 polling。

`tx_complete():189` 调 `vq.poll_used(token,len)`。实际 split_virtqueue.poll_used():339 先做 read barrier，再读共享 Host backing 的 used.idx。等于 last_used_idx 时返回 0；有新完成才取 `{used_id,used_len}` 并查 token_map。

成功取回原 pkt token 后，queue 删除 token、释放可能的 indirect table，并把 descriptor 链放回 free list。本次两个 descriptor 归还，free count 回到 256；last_used_idx 加到 1。used_id 不在 token_map 时报告错误并返回 0，链回收中检测到循环也会报错，不应把错误日志与函数返回值混为一个“成功”判据。

随后 ops 根据完成 packet 数退休普通 DMA 记录。普通带 header 的队列每个 packet 对应两条记录，因此本次从 tx_dma_map[qid] pop header 和 payload，调用 `retire_normal_dma_record():153`，逐条 `iommu.unmap_for_host()` 然后 `mem.free()`。空记录队列被删除，但 ring 的三段 backing 和映射仍保留，供下次提交复用；整设备 reset 才拆除它们。

fixture 的 `ok=(completed.size()!=0)` 只说明取到了非零完成，并不保证一般情况下达到 max_budget，也不直接比较完成对象是否和原 pkt 逐字段相等。本测试在第 113～123 行另外断言 count=1、objects=1，以及 notify、DMA read/write、completion、interrupt 计数有增长。

11.6 monitor、scoreboard 与测试各证明什么

被动路径为 PCIe RC/EP monitor.tlp_ap → per-Function observer adapter.write → virtio_monitor.observe_* → FIFO/event/SVA/txn_ap。adapter 用 frozen identity、requester BDF 和 owned BAR 区间路由，先把 MSI-X 与 MMIO 分出来，再把本 Function 的其它设备内存请求归为 DMA；它不替设备做业务搬运。

virtio_monitor.observe_dma():189 在 DMA window 已配置且检查开启时验证范围、非零大小和地址回绕。`observe_queue_notify():275` 与 `observe_interrupt():218` 检查通知/完成信用等语义，`publish_event():498` 按事件写 FIFO、触发 event、广播 txn，错误还通过 err_ap 报告。所有这些是观察，不是写 used 的生产者。

scoreboard.write():79 对 is_monitor_event 单独计数并处理部分检查，然后直接 return。当前黄金测试直接调用 ops，未通过 `VIO_TXN_SEND_PKTS/WAIT_PKTS` 构造一对 expected/actual 业务事务。因此不能声称 scoreboard 在这个 TX 测试中完成了 16 B wire payload 的逐字节比对。

scoreboard 的 `check_dma_access():337` 当前只拒绝 size=0；名字和旧注释比实际实现更宽，不能据此宣称它独立验证了每条 IOMMU 权限。DMA 权限的真实拒绝点在 port/IOMMU；monitor 另负责已配置窗口的被动检查。`check_notification():321` 主要记录日志，不能把它当成完整中断协议证明。

当前证据来源	能说明什么	不能单独说明什么
TX 测试完成数与计数断言	请求穿过 notify/DMA/used/IRQ 路径	全套 Ethernet/offload 正确性
Device DMA 与 used	真正消费共享 backing 并发布完成	原 test 对象的所有字段都被比较
Observer / monitor / SVA	地址、状态、通知等已实现契约被观察检查	自动拥有所有 payload expected
Scoreboard 业务事务分支	有 SEND/WAIT 事务时可做 expected/actual 匹配	直接 ops 主线必然走过该分支
下一章 RX test	实际内存读回形成的帧与输入字节一致	自动代表所有协议/异常场景已覆盖

这不是降低主线价值，而是明确“闭环在哪里产生、哪些条件由谁断言”，避免仅看到 scoreboard 被创建就认为所有七类检查都已覆盖了当前业务。

12. 具体业务：四帧 RX 从网络入口回到 packet_item

12.1 这是独立测试，网络侧字节必须另有来源

入口是 virtio_real_driver_rx_test.run_phase():28。它的 build_phase():23 新建同类 fixture，从新一轮 UVM 生命周期开始；不是上一章 TX 测试完成后继续处理回环帧。run 先 raise objection，检查 build_flow，经 `rx_queue_id_for_pair(0,rx_queue_id,why)` 查询冻结资源映射，再初始化 driver 并建立一条 split256 RX 队列。默认 pair0 对应 RX0，但查询结果才是本次使用的 qid。

第 56 行显式检查 `execution_profile().has_model_device()`。不是 MODEL 就以 `REAL_DUT_RX_SOURCE_UNAVAILABLE` fatal 结束：当前测试没有平台网络入口 callback，RC Host-memory responder 也只回答设备 DMA，不能制造网络接收帧。设置 REAL_DUT 模式不会自动把模型入包变成 RTL 端口激励。

队列建好后，测试记录 notify、DMA write、completion、interrupt 四项设备计数，再构造四个外部 `packet_item`，约束 `pkt_kind==ETH_IPV4_UDP`、`pkt_len inside {[128:512]}`。随机化失败立即 fatal。成功后 `pkt.pkt.do_pack()`，保存 expected 句柄，调用 fixture.inject_rx。这里的 `pkt_len` 是外部报文对象的生成参数，源码中实际传递的是 `pkt.pkt.raw_data`；不能直接把 128..512 当作已证明的准确 wire 字节数。精确长度应取 pack 后的字节队列大小，本章以 `F_i` 表示第 i 帧的实际字节数。

12.2 inject 只入设备 backlog，不直接填 Host buffer

fixture.inject_rx():959 接收 qid、packet 对象和 ref ok。它先置 ok=0，要求 MODEL device 存在、packet 非空、qid 为偶数且 `virtio_net_packet_adapter::pack` 成功；失败报告 UVM_ERROR。成功时复制 wire bytes 到 `bit[7:0] frame[$]`，调用 `dev.inject_rx(queue_id/2,frame)` 后置 ok=1。

继承自 vdev_base.inject_rx():282 的实现把字节队列存入 `rx_backlog[pair]`，并 `schedule(2*pair)` 唤醒 RX 队列。这个函数不写 Host buffer，也不创建 used 项；没有“帧已经成功 DMA 到 Host”的返回值。fixture 的 ok 只说明通过包装层校验并把帧交给设备。

四帧全部先进入 backlog，然后测试才执行 refill。即使 worker 先获得调度，RX ring 尚无 avail 时 `fetch` 返回 got=0，帧仍留在 backlog。refill 发出的 notify 再调度同一 RX 队列，`service_queue():491` 检查 epoch、queue enable、DRIVER_OK、broken 和 backlog，之后才走 fetch→chain_ok→consume→push_used。这正是测试有意验证的“帧先到、可写 buffer 后到”。

12.3 refill 发布四个可写 buffer，容量与有效长度分开算

fixture.refill_rx():984 查找 selected Function 和 ops，调用 `ops.rx_refill(queue_id,count)` 后设置 ok=1。它不检查内部实际填了多少，也不额外检查 flow started；正式测试通过前序初始化保证调用时机。

生产入口 rx_refill():238 的第三参数 `buf_size=0` 表示使用默认容量：先尝试 queue_program 中的 RX 容量，仍为 0 则取 header 长度+1514。显式非零参数优先于这两者。标准主线协商 CSUM|VERSION_1，header 为 12B，所以 descriptor 容量为 1526B；分配与映射大小 DWORD 向上取整为 1528B。

量	单 buffer 主线值	作用
Virtio header 长度	12B	VERSION_1 带来字段布局，不代表 MRG_RXBUF 已协商
Descriptor 可写长度	12+1514=1526B	设备可用容量，不是本次已收长度
GPA 申请及 IOVA 映射长度	1528B	DWORD 对齐的 backing，权限 DMA_FROM_DEVICE
实际 used_len	正常时 12+F_i	设备写回的 header 与帧总长

每次循环创建一份独立 buffer：DMA 集合 alloc→清零→map(DMA_FROM_DEVICE)，生成一个 SG entry `{addr=I_buf,len=1526}`，调用 `vq.add_buf(sgs,0,1,null,0)`。n_out=0、n_in=1 说明设备可写；token=null 说明接收前没有一个可供“归还”的原输入 packet 句柄。split 队列仍在 token_map 登记该 head，只是其值为空，因此 poll_used 检查的是键存在，不是 token 非空。

新队列成功填四份时 head 依次 0、1、2、3，每份一个 WRITE descriptor、一个 avail 条目，free count 由 256 到 252。GPA 为随机分配，不能画成四份连续 buffer。每份 add_buf 成功后经 adopt_dma_set 把所有权转给 `rx_dma_map[qid]`；整个循环末尾有成功提交且 needs_notification 时才统一 kick。清零 buffer 是驱动准备可写内存，不是伪造设备数据。

队列不存在会早返回；ring 满、Host 分配失败、IOMMU 映射失败或 add_buf 失败都可能提前结束。映射/add_buf 失败回滚当次 DMA 集合，此前成功的 buffer 仍在 RX 表中等待完成。task 没有 filled 输出，fixture.ok=1 不能代替“确实填了四份”的断言；test 后续要求收到恰好四帧才完成这个验证。

12.4 设备写 header、payload 和 used，used_len 说明实际写入多少

`consume` 的 RX 分支调用 vdev_virtio_pci.rx_frame():820。它取并弹出该 pair 的第一帧，构造全零 header 后设置 `num_buffers=1`，通过 encode_hdr 生成 12B，再追加帧字节。主线 header 的偏移 10（从 0 计数，即第 11 字节）因此是 1，而不是 TX 主线的全零 header。

设备先累加链内 descriptor 的容量 room。若 `12+F_i > room`，报告 VDEV_RX warning，丢弃已经出 backlog 的帧，保留 ok=1、used_len=0 返回；service_queue 随后可以把 buffer 以 len0 归还。不能把“有 used 完成”单独看成“收到完整帧”。若容量足够，逐 descriptor 取相应片段，经 `dma_wr(chain[i].addr,part,ok)` 写入设备可见 IOVA。port 验证 FROM_DEVICE 写权限并翻译到共享 GPA，再发 EP-originated PCIe 写，具体传输边界沿用第 11 章。

任一 DMA 写失败令 ok=0 并返回，本轮不会正常 push_used；被弹出的帧也不会由本函数自动放回 backlog。全部写成功后发布链/header 和 RX 字节事件，设置 `used_len=payload.size()`，即 12+F_i。然后 push_used 写 `{head,used_len}`、推进 used.idx 并决定中断。四帧正常接收各产生一个完成；设备不把 padding 的两字节算进 used_len，也不会把整份 1526B 容量一律上报为有效数据。

这个 MODEL 实现一次处理一条 descriptor 链，放不下就 len0 归还，并没有在此自动跨多条独立 avail 链合包。驱动侧支持 MRG_RXBUF 的代码存在，不表示默认设备模型或当前测试覆盖了该场景。

12.5 Host 按 used 长度复制字节，再退休每份 DMA

fixture.receive_rx():1007 先清 received 队列与 count，检查 Function/ops，再有限次调用 `ops.rx_receive(queue_id,received,budget)`。累计数量达到 budget 即停止，没有新增对象才等 1ns；循环耗尽也返回已有结果。最后 `received_count=received.size()`、`ok=(received_count!=0)`，所以一般调用者必须另查 count，不能把非空等同于收到 budget 份。

底层 rx_receive_frames():325 的输入是 qid 与 max_budget，frames 为追加式 ref 输出。它取得 vq、算 header 长度，循环 poll_used。split.poll_used 从共享 Host ring 读取完成、取回 null token 并归还 descriptor；之后 ops 从 `rx_dma_map[qid]` 按提交顺序 pop 一份 normal DMA record，用其中 GPA 读取 used_len 字节。

当前实现用 FIFO 所有权表配合设备按提交次序完成的 MODEL 主线，不是在这里拿 used_id 做通用乱序 DMA 映射查找。used 有完成但 ownership 表为空会报错停止。读回长度小于 header 时报告短包、退休当前 record 并停止；例如设备容量不足归还 len0，不会成为一份合法 packet。

正常路径 unpack 前 12B header，剩余字节进入 `frame.payload`，然后 retire 当前 record：按域 unmap IOVA，再 free GPA。数据已经复制进 frame，DMA backing 随后释放不会丢失返回内容。其合包数计算是：没有协商 MRG_RXBUF，或 header.num_buffers 为 0，都按 1 处理。VERSION_1 虽然携带 num_buffers 字段，却不能单独让该字段触发多 buffer 拼接。

如果另一测试真的协商 MRG_RXBUF，后续 buffer 会继续 poll_used、复制全部续接字节并分别退休 record。缺续接 buffer 时会报错跳出合包循环，但本实现仍可能把已组装的部分 frame 追加到 frames；不能声称“解析成功就保证完整合包”。本主线无 MRG 且每帧一 buffer，不经过该异常分支。队列 RX 记录全部耗尽后删除该映射表项，ring 资源留到最终 reset 再拆。

12.6 从 frame 到 packet_item，逐字节比较才闭合 RX 内容证据

ops.rx_receive():419 对每个 frame 调用 virtio_net_packet_adapter.unpack():41。adapter 拒绝空字节，创建新的 packet_item 及 packet payload 对象，调用外部 payload.unpack，再按 raw_data 长度是否一致返回成功。这不是把 inject 传入的 expected 对象直接还给测试。

unpack 成功才把新对象加入 received；失败且 frame.token 非空才回退返回 token。本 RX refill 的 token 为 null，解析失败便不会增加 received 数量。adapter 的长度判定本身也不等于验证所有 IP/UDP 字段或校验和合法性；完整内容比对由测试下一步完成。

test 首先要求 count 和 received.size 都等于 4，再逐项 cast 为 packet_item，调用 adapter.compare():57。compare 将 expected 和 actual 通过 pack 转成 wire 字节，先比长度再逐 byte 比较；空对象、无法 pack、长度或任意字节不同都失败。它没有依赖 packet_item.do_compare 的字段选择，也没有把 Virtio header 一起当以太帧比较。

因此本测试的内容证据链为：输入 raw_data → backlog 帧 → 设备 DMA 写 buffer → Host 按 used_len 读回 → 去 12B header → 新 packet_item.raw_data → expected/actual 逐字节一致。测试另查 notify、DMA write、completion 和 interrupt 计数均增加，最后执行 reset_and_teardown。通过这些断言仍不表示所有 checksum、RSS、GSO、多队列、mergeable 或 REAL_DUT 接入场景都已覆盖。

12.7 dataplane engines 是另一层调用入口，不是第二个 DMA 所有者

Function 创建的 dataplane wrapper 已有 TX/RX engine，但这个 RX 测试直接通过 fixture 调用 ops，并没有调用 rx_engine.receive_packets。引擎对照能解释为何项目保留 wrapper 而仍强调 ops 作为统一资源边界。

入口与位置	做什么	参数与边界
`src/dataplane/virtio_rx_engine.sv:88 refill_buffers`	调 `ops.rx_refill(qid,count,get_rx_buf_size())`	显式传 engine 容量；缺 ops/manager/queue 报错；不独占 DMA
同文件:`107 receive_packets`	调 `rx_receive_frames`，可选 checksum 检查、建 packet 并累计统计	qid、budget、ref received；缺上下文早返回，checksum 失败告警；free count 低于 threshold 再补 buffer
同文件:`64 build_received_packet`	adapter.unpack 成功返回 packet_item，否则创建 wrapper	wrapper 保留 payload/header/实际长度，供错误测试检查

入口与位置	做什么	参数与边界
`src/dataplane/virtio_tx_engine.sv:55 submit_packet`	从字节推导 header，再调同一个 ops.tx_submit	qid、pkt、indirect、ref desc_id；空数据拒绝，不走 ops 的空 payload64B 兜底
`src/dataplane/virtio_net_dataplane.sv:92 bind_ops`	两 engine 借用同一个 ops	仅接线，不分配 ring 或报文

源码：RX engine、TX engine。

RX engine 只在协商 GUEST_CSUM 时调用 offload.verify_rx_csum；默认主线协商的是 CSUM，不是 GUEST_CSUM。engine 还在解析失败时返回 wrapper，而直接 ops.rx_receive 只在 token 非空时回退；两条 API 返回语义不能混写。自动补充条件实际是 `vq.get_free_count() < refill_threshold`，threshold=0 关闭该分支，不应泛化成“每收到一个包自动补一个”。这些 wrapper 承担转换、策略和统计，buffer 分配、IOMMU、descriptor、notify 与退休仍委托同一个 ops。

13 把各层连成一次完整运行

13.1 从 run_test 到 flow ready

前面各章按对象和数据结构拆开阅读；这一章按时间重新串联。主线仍是 `virtio_real_driver_flow_test` 的 MODEL 运行，RX 放在独立小节，因为它由另一个 test 创建全新 fixture，不是 TX run 的下一段。

顶层 `virtio_tb_top.sv:71` 调用 `run_test()`，测试名由仿真参数 `+UVM_TESTNAME` 选择。顶层另有 250 MHz 时钟、100 ns 平台复位以及 30 s 仿真时间兜底。`run_test` 前的 `#0` 给生成的接口注册过程一次执行机会；它不等待业务初始化。测试 build phase 创建 fixture，此后 UVM 各 phase 才逐步产生可运行环境。

阶段	发生的业务动作	尚未完成的动作
test.build	工厂创建 `flow`	未做 feature 协商
fixture.build	提交 DPU 配置请求，建立共享内存和各环境；MODEL 创建 device	未分配业务 ring
DPU/环境解析	生成冻结的设备/资源视图，按 service 建 Function	未写 DRIVER_OK
fixture.connect	绑定 PCIe RC/EP 观察与事务路径，接 MODEL device port	未提交包

阶段	发生的业务动作	尚未完成的动作
fixture.end_of_elaboration	从 snapshot 解析 AF/BDF/BAR，配置并挂接模型，取共享 Host memory	未产生 used completion
test.run / build_flow	检查冻结对、路径、端点与共享句柄 ready	尚需执行 driver flow

这里有两个名字容易误读。`configure_devices` 创建的是设备模型的配置空间/寄存器初始状态，并赋予 frozen BDF；Host 驱动后面仍要经 PCIe 配置事务编程 BAR。`build_flow` 则是准备条件的验收函数：如果没有 endpoint ready、snapshot 对不匹配、PCIe 路径没接通、REAL_DUT 的双向链路/Host memory 服务缺失，返回 0 并填写 why。它本身不会偷偷完成 `start_driver_flow`。

源码：run_test():71、build_phase():99、connect_phase():263、end_of_elaboration_phase():307、build_flow():412。

13.2 golden TX 的顺序与证据

test.run 首先 raise objection，使业务 task 没有完成时 UVM 不提前结束。`build_flow` 成功后，测试检查 snapshot frozen、BDF 非零、三对 BAR、共享 Host memory 和 transport 环境句柄，再通过 resource client 映射取得 pair 0 的 TX local queue ID。默认是 TX1，调用链仍然使用查询结果。

接下来可按下列时间线定位断点。这里的箭头表示先后依赖，不代表同一仿真时刻完成；PCIe completion、device worker 和观察事件可能异步推进。

1. `start_driver_flow(ok)`：等待协议观察接口退出平台复位；发现/编程 BAR，开 MSE/BME；执行 verified reset；写 ACKNOWLEDGE、DRIVER；协商 CSUM|VERSION_1；写后读回 FEATURES_OK；检查四条队列容量；配置 MSI-X；写后读回 DRIVER_OK。完成后 fixture 标记 flow started，FSM 视图为 READY。
2. `setup_queue(tx_queue_id,256,VQ_SPLIT,ok)`：生产 ops 创建一条 split 队列，分配三个 Host 区域，映射为设备 IOVA，写地址/向量/notify offset 并 enable；fixture 额外读 queue_enable。golden 没有执行 `full_init`，因此不会再建一组重复 ring。
3. 构造 `virtio_real_driver_flow_packet`：payload 固定为 `de ad be ef 01 23 45 67 89 ab cd ef 10 20 30 40`，header 全零；记录模型 notify、DMA read、DMA write、completion、interrupt 五个计数基线。
4. `submit_tx(...,use_indirect=0,desc_id,ok)`：生产 TX 操作分配和映射 header/payload，经队列发布 descriptor 与 avail，按条件 kick。成功描述符 ID 不等于全 1；fixture 不直接改 ring 来模拟设备。
5. MODEL 经收到的通知处理队列，读取 descriptor 和数据，执行设备侧 TX 行为，再经设备 DMA 写 used 并产生中断。详情见第 10～12 章；测试此时没有“帮设备写完”的快捷操作。
6. `complete_tx(...,max_budget=1,completed_count,ok)`：反复调用生产 `tx_complete`，直到出现完成或等待界限耗尽。test 要求 ok、count=1、对象数组大小=1，并要求五项设备计数各自增加。

7. `reset_and_teardown(ok)`：等待观察事件可被检查，确认设备 reset，释放 ring/DMA，检查 manager 清空与泄漏报告，再 drop objection。

本测试的正向证据是“正式 TX 提交和回收、设备通知/DMA/used/中断计数发生”。它没有取出 wire-side frame 与 16 字节 payload 做逐字节比较；也没有把 returned token 的数据逐字节重读核验。不能把 completion count=1 宣传成任意 offload、分段、wire payload 内容都已验证。全零 header 也不意味着 CSUM feature 未协商，只表示本次报文不请求 checksum offload。

源码：run_phase():40、start_driver_flow():736、setup_queue():830、complete_tx():918。

13.3 独立 RX run：先到帧，后有可写 buffer

选择 `virtio_real_driver_rx_test` 会从 UVM build 重新开始。它同样通过 frozen placement 查询 pair 0 的 RX local queue ID，初始化 driver flow，再单独建立 split256 RX ring。该测试显式要求 MODEL；REAL_DUT 分支在缺少平台 RX ingress callback 时 fatal，不会自动拿模型代替 RTL。

RX test 记录 notify、DMA write、completion、interrupt 基线，然后生成四个 `packet_item`。每个对象约束 `pkt_kind==ETH_IPV4_UDP`、`pkt_len` 在 128..512，randomize 成功后执行 `pkt.pkt.do_pack()`，保存为 expected，并调用 `flow.inject_rx` 送入设备网络侧。四帧全部先入设备等待队列，此时还没有驱动发布的 RX writable buffer；这样特意验证设备会等资源，而不是测试预写 payload。

随后 `refill_rx(rx_queue_id,4,ok)` 调用生产 ops 分配四份空 buffer、做 FROM_DEVICE 映射、构造可写 descriptor、发布 avail 与通知。MODEL 自己消费已排队的帧并写入 header、payload、used。`receive_rx(...,budget=4,received,received_count,ok)` 反复调用生产接收接口。test 要求 received_count 与数组大小都精确等于 4；每项必须能 cast 为 `packet_item`，且 `virtio_net_packet_adapter::compare(expected[i],actual)` 成功。最后检查四项计数增加，再 reset/teardown。

因此 RX 的断言比 golden TX 多一个明确的 packet compare 边界：adapter 将 expected/actual 都 pack 成字节序列，先比长度，再逐字节比较。它覆盖本次随机生成帧在 adapter 契约下的内容一致性，不等于覆盖所有 IPv4/UDP checksum 错误、GSO、mergeable buffer、RSS 或 MQ；是否覆盖这些行为，要看各专项测试的 feature、报文和断言。

还要区分三个 helper 的 ok：`refill_rx` 的 ok=1 只表示找到有效 ops 并调用返回；`rx_refill` 内部可能因 ring 满、内存不足或映射失败提前结束。`complete_tx` 的 ok 表示至少收到一个完成，不保证 max_budget 项。`receive_rx` 的 ok 表示至少收到一个包，不保证 budget 项。请求数是否满足，必须看调用者对 count 和数组长度的断言，本 RX test 正是这样做的。

源码：RX run_phase():28、refill_rx():984、receive_rx():1007、compare():57。

14 生命周期、故障定位和运行入口

14.1 reset 的第一件事是证明设备停下

queue manager 的 `destroy_all()` 只会释放 Host ring 对象，不能证明设备已经停止使用这些地址。正式收尾调用 `device_reset_verified(ref reset_complete)`，先检查 transport、manager、IOMMU 以及待回收普通 DMA 所需 memory 上下文，再通过配置 backend 请求 reset 并等待确认。标准 backend 最终写 status=0 并轮询回读 0。

未确认时，它置 `queue_recovery_pending=1`、`ctrl_recovery_pending=1`，返回失败并保留旧 ring、IOVA 和 DMA backing。后续建队/control 提交会受恢复门禁阻止。保留资源是因为设备仍可能访问；这时不应为了让 leak check 好看而强行 free。上下文不完整也进入同一保守门禁，不能把缺少 handle 当成已经 reset。

只有设备 reset 确认后，才执行以下释放链：detach 未完成 buffer 的 descriptor 所有权；逐条撤销 ring IOVA 并清 `queue_programs`；retire 尚归 Host 持有的普通 TX/RX DMA 记录；回收迁移恢复 payload 与 control quarantine；最后 `vq_mgr.destroy_all()` 释放 ring 的 Host 分配并删除对象。成功路径清控制队列身份、negotiated features 与两个 recovery pending 标记。完成路径已经回收的 DMA 记录不应再由 reset 重复释放，所以 ownership 表是这段实现的核心。

fixture 的 `reset_and_teardown` 在此之前关闭可选 Host-memory responder，并在 FSM_RUNNING 时停止数据面 worker。它调用 `drain_protocol_events` 等待 MSI-X 发布延迟、staged protocol 事件消耗并再等两拍，避免立即复位使尚未检查的事件被清除。该 drain task 超时会报告 UVM_ERROR，但没有状态输出阻止后续 reset；不能把最终 helper ok 当成“整轮绝无观察错误”，还要检查 UVM summary。

verified reset 失败时 fixture 返回 ok=0；成功后检查队列数为零并调用 IOMMU/Host memory leak_check，最后将 FSM 视图设 IDLE、flow started 清零。leak_check 的报告也不都被折算为 helper 的 ok，所以严格日志检查仍是完成条件的一部分。

源码：device_reset_verified():14、drain_protocol_events():1045、reset_and_teardown():1075。

14.2 按最早断开的边界定位

看到“没有收到包”时，先找最早失去证据的位置，不要直接扩大轮询超时。下面每行只给第一组定位入口；修复必须遵守 snapshot、外部依赖和资源所有权边界。

现象	首先核对	可区分的问题
build_flow 返回 0	why、frozen snapshot 对、endpoint/path ready	平台接线与业务初始化失败

现象	首先核对	可区分的问题
discovery fatal	BAR lease/role、capability walk、Command 位	身份/布局错误与队列错误
FEATURES_OK 被清	device mask、driver mask、设备 status	配置愿望与实际协商
setup_queue 返回 0	重复 ID、recovery pending、Q_SIZE、域、映射/向量	上一生命周期未结束与资源不足
有 avail，无 notify	suppression、notify offset、kick 日志	合法抑制与地址/队列错误
有 notify，无 DMA	DRIVER_OK、queue enable、设备 error、BME	未激活队列与设备拒绝 descriptor
有 DMA，无完成	header/chain 解析、IOVA 权限、used 发布	数据访问失败与完成写失败
有 used，Host 没回收	ring 格式、used index/wrap、预算与 token	设备完成与驱动消费失败
有 completion，无中断	MSI-X Enable/Mask、vector 映射、suppression	polling/抑制与真正丢中断
reset 后资源残留	reset_complete、pending 标记、ownership 表	安全保留与成功 reset 后漏回收

地址问题至少带上 host、BDF、queue ID、GPA、IOVA、长度与方向。仅记录一个十六进制地址通常不足以定位：同一 Host 的共享 backing、不同 BDF 的设备 DMA 域以及 BAR MMIO 窗口有不同意义。随机地址策略下也不应靠“这次地址没有连续”判错。

默认 flow 轮询界限是 1000 次，每次无完成时推进 1 ns，可由 `+VIRTIO_FLOW_TIMEOUT_NS` 调整，0 被归一为 1。它与 PCIe completion timeout、wait policy 的 reset 超时、顶层 30 s 仿真时间兜底、strict regression 的墙钟超时不是同一计时器。调整哪一个应取决于哪个等待先失败，不能用一个巨大上限掩盖没有接通的 EP→RC 路径。

14.3 MODEL 与 REAL_DUT 的替换边界

MODEL 的 execution profile 选择 TLM_MODE、active EP 和设备模型；fixture 安装 completion adapter/factory override，通过 `vdev_uvm_ep_port` 将设备配置/MMIO、DMA 与 MSI-X 接到 PCIe 环境。它仍然要求正式 driver 写寄存器与 ring；“MODEL”不等于测试直接造一个 used 条目。

REAL_DUT 的 profile 不创建 `vdev_virtio_pci`，选择 SV_IF_MODE、passive EP，关闭自动设备应答。配置/BAR/device 行为应来自 RTL。fixture 要求两条物理方向：RC→EP 与 EP→RC；单个 `pcie_tl_if` 只有一个有效方向，不能同一 bundle 同时充当双向链路。Virtio monitor 保留观察功能，却不会因此获得替 RTL 完成请求的权限。

EP 发起 Host memory DMA 还需要 RC 服务。可以由外部 FULL-VIP/backend bridge 提供，也可显式启用 fixture 的 Host-memory responder；两者同时启用会冲突，存在重复 Completion 风险，`build_flow` 会拒绝。该 responder 只服务共享 Host memory 的 MemRd/MemWr，不是 Virtio device model，不会构造网络 ingress，也不会替设备推进 used ring。

REAL_DUT TX 测试使用生产提交/回收接口，并检查被动 monitor 的 notify/DMA/interrupt 事件 FIFO 增长，不能去取 MODEL 的 device counters。REAL_DUT RX 在当前独立 RX test 中缺少平台 ingress callback，明确以 `REAL_DUT_RX_SOURCE_UNAVAILABLE` 终止。仅设置执行模式 plusarg 无法让默认顶层凭空出现 RTL、双向 VIF 或网络入口。

源码：execution profile、REAL_DUT readiness:432。

14.4 如何复现，以及本文没有宣称什么

本文为源码阅读与文档交付，没有执行 VCS，没有新增某个测试 PASS 的结论。所有 VCS 验证必须在项目指定的 `10.11.10.53` 仿真主机进行，使用加载 `.bashrc` 中 VCS 路径及 license 环境的 bash 登录环境；认证信息不属于本文内容。应先在该主机核对 checkout 基线与外部依赖，不能在阅读机的任意环境直接宣称复现。

`scripts/check_deps.sh` 是编译前置门，检查外部依赖路径、remote 与版本契约。本文基线固定 `dpu_common=4d739965eb47d90b47cc048fc71fd8d7d76a77ca`、`host_mem=35ec087014744ec85cf6c0fe17e1f7118ee7a7b7`；其余依赖按脚本各自规则校验。本次阅读环境的 dpu_common HEAD 为 `311a9d…`，与 pin 不同，因此不能宣称依赖检查已通过。本文没有修改依赖仓库来消除差异；未来复现应准备符合项目契约的外部 checkout，再过门。

在仿真主机的正确仓库工作目录、依赖环境与 VCS 环境已配置后，两个主线入口是：

```
TEST=virtio_real_driver_flow_test bash scripts/vcs.sh
TEST=virtio_real_driver_rx_test bash scripts/vcs.sh
```

这是两次独立运行。`scripts/vcs.sh` 使用环境变量 TEST 选择 maintained test，唯一可选位置参数为 `--compile-only`；把测试名放在位置参数会被拒绝。脚本先检查 VCS 和 TEST，再调用依赖检查、编译固定 filelist，最后执行 `build/simv +UVM_TESTNAME=...`。它不把任意额外命令行参数透传给 simulator。

需要详细日志时可以设置 `UVM_VERBOSITY=UVM_HIGH`。需要自行给 simv 传递执行模式等 plusargs 时，应在同一受控环境先用合法 TEST 做 `--compile-only`，再直接运行已生成的 simv；REAL_DUT 还必须使用实际接好 RTL/VIF 的平台，不能把下面的 MODEL 编译入口误作 RTL 接入完整方案。

```
TEST=virtio_real_driver_flow_test bash scripts/vcs.sh --compile-only
build/simv +UVM_TESTNAME=virtio_real_driver_flow_test +UVM_VERBOSITY=UVM_HIGH
```

“进程退出为 0”不等于验证成功。`scripts/strict_log_check.sh` 分 compile/sim 模式，拒绝 VCS warning/error；sim 还要求恰好一份 UVM summary，WARNING/ERROR/FATAL 各一行且都为 0，并拒绝 Host memory leak、PCIe completion 字段 mismatch 和非零 scoreboard mismatch。`strict_regression.sh` 统一编译后逐个运行 maintained tests，另加默认 180 s 墙钟超时并对每份日志执行该检查。本次未运行这些脚本；这里解释可复现入口，不把脚本存在当成已有测试证据。

源码：vcs.sh、check_deps.sh、strict_log_check.sh。

附录 A 按阅读目的查文件

本附录只提供索引；资源与初始化的逐函数参数表见第 9 章，TX/RX 数据动作见第 10～12 章。所有路径相对仓库根目录。

阅读目的	文件或目录	优先查找的入口
UVM 从哪里启动	`virtio_net_vip/tests/virtio_tb_top.sv`	`run_test`
最短 TX 业务契约	`virtio_net_vip/tests/virtio_real_driver_flow_test.sv`	`run_phase`
随机网络帧 RX 对比	`virtio_net_vip/tests/virtio_real_driver_rx_test.sv`	`run_phase`
环境准备与业务包装	`virtio_net_vip/tests/virtio_real_driver_flow_fixture.sv`	phase、submit/receive/reset helpers
Function 边界	`virtio_net_vip/src/sriov/`	resource client、function instance
驱动动作的真实实现	`virtio_net_vip/src/agent/atomic_ops/`	lifecycle、queue、datapath 文件
自动状态机的另一入口	`virtio_net_vip/src/agent/virtio_auto_fsm.sv`	full_init、start/stop_dataplane
环格式与 Host ring 访问	`virtio_net_vip/src/virtqueue/`	alloc_rings、add_buf、get_buf
配置/MMIO 与 notify	`virtio_net_vip/src/transport/`	discovery、backend、transport
真正设备侧处理	`virtio_net_vip/src/devmodel/vdev_virtio_pci.sv`	service_queue、consume、push_used
设备 PCIe 端口	`virtio_net_vip/src/devmodel/vdev_uvm_ep_port.sv`	serve、dma_read/write、raise_msix
packet 对象转换	`virtio_net_vip/src/dataplane/virtio_net_packet_adapter.sv`	pack、unpack、compare
地址权限与域	`virtio_net_vip/src/iommu/virtio_iommu_model.sv`	map_for_host、unmap_for_host
私有协议独立实例	`examples/dpu_priv_ring/`	driver flow 与 contract test
可运行测试名单/判定	`scripts/`	test_manifest、vcs、strict_log_check

设备模型导航：service_queue():491、push_used():948、DMA 端口:192。

附录 B 术语与阅读自检

B.1 本文术语

术语	本项目中应如何理解
frozen snapshot	DPU 权威解析完成的设备/资源视图，Virtio 只消费
Function / BDF	PCIe 功能及其 bus/device/function 身份；完整域还含 host/segment
service	附属于 Function 的 VIO-net 服务，是资源绑定和行为配置的键
qpair	一对 RX/TX 资源；global qpair ID 不等于 local queue ID
GPA	Host memory backing 地址，Host 分配与读写使用
IOVA	设备可见 DMA 地址，受 host/BDF 映射和权限限制
BAR	PCIe 寄存器/功能窗口，不能当作 ring backing
SG / descriptor	一组缓冲段及其设备可读描述，方向以设备视角解释
token	随提交保留的 Host 软件对象身份；不等于设备 wire payload
avail / used	split 中的驱动发布与设备完成索引/条目
packed wrap	packed 描述符归属随环回绕改变的判定状态
notify / kick	Host 通知设备检查已发布队列
MSI-X	设备向 Host 发布的中断消息，受表项/向量/mask 控制
verified reset	确认设备停止旧生命周期后才能回收 DMA 的复位路径
recovery pending	未能确认安全回收时阻止新建队列及 control VQ 提交的状态门禁
MODEL / REAL_DUT	本地设备模型执行 / RTL 执行，不能混用完成来源

B.2 能回答这些问题，再继续改业务

- pair 0 的 TX 队列号从哪一条资源映射取得？为什么不能从 global qpair ID 推导？答题定位：第 6 章的 resource client 与 fixture 查询。
- 全 1 driver feature mask 为什么没有让 golden 使用 packed、MQ 或 mergeable RX？定位：第 7 章的设备广告与交集。
- golden 哪个函数真正分配 ring？它为什么没有执行 full_init？定位：第 8 章的 setup_queue 和第 13 章时间线。
- 描述符中的地址、common config 中的 ring 地址、Host mem 读写地址分别属于哪个地址域？定位：第 8 章 GPA/IOVA。
- packed 一次 Host 分配为什么仍有三份 IOVA 区域？定位：get_ring_regions 与 ops 的映射粒度。
- 已有两对 Host ring 为什么还不能直接启动标准 MQ 数据面？定位：mq_pairs_committed 与 control VQ 命令。
- RX test 在没有可写 buffer 时把四帧放在哪里？谁在 refill 后写 payload 和 used？定位：第 13.3 节及设备 rx_frame。
- helper ok=1、count 达标、packet compare 成功和严格日志通过，各自排除了哪一类失败？定位：第 13 章及第 14.4 节。
- 如果 status=0 的回读一直未完成，应该释放 ring 还是保留？什么门禁阻止新队列混进旧生命周期？定位：device_reset_verified。
- REAL_DUT Host-memory responder 为什么不能解决 RX 网络入口缺失？定位：第 14.3 节的组件职责。
- golden TX 的计数都增加，是否证明线上 16 字节内容完全正确？定位：第 13.2 节测试断言边界。
- 新跑的一份日志出现 UVM_WARNING，但进程退出码为 0，能否登记 strict PASS？定位：strict_log_check 的 summary 规则。